



PRoNTTo Manual

The *PRoNTTo* Development Group

John Ashburner
Carlton Chu
Andre Marquand
Janaina Mourao-Miranda
Christophe Phillips
Jonas Richiardi
Jane Rondina
Maria J. Rosa
Jessica Schrouff

Machine Learning & Neuroimaging Laboratory
Centre for Computational Statistics and Machine Learning
Computer Science department, UCL
Malet Place, London WC1E 6BT, UK
January 30, 2012

<http://www.mlnl.cs.ucl.ac.uk/pronto>

Contents

1	Introduction	7
1.1	Goals	7
1.2	Main contributors	8
1.3	Acknowledgements	9
I	Graphic User Interface	11
2	Data & design	13
2.1	Introduction	13
2.2	Methods	13
2.3	Graphical User interface	14
2.3.1	PRT directory	14
2.3.2	Groups	15
2.3.3	Subjects	15
2.3.4	Modalities	15
2.3.5	Masks	17
2.3.6	Review	19
2.3.7	Load, Save and Quit	19
2.4	Matlab batch interface	21
3	Prepare feature set	23
3.1	Introduction	23
3.2	Methods and resources	23
3.3	Graphical User interfaces	25
3.4	Matlab batch interface	26
4	Model & CV scheme specification	29
5	Mode & feature weights estimation	31
6	Results display	33
II	Batching system	35
7	Data & Design	37
7.1	Directory	37
7.2	Groups	37
7.2.1	Group	37
7.3	Masks	39
7.3.1	Modality	39
7.4	HRF overlap	39
7.5	HRF delay	39
7.6	Review	40

8 Feature set / Kernel	41
8.1 Load PRT.mat	41
8.2 Name	41
8.3 Modalities	41
8.3.1 Modality	41
9 Specify model	43
9.1 Load PRT.mat	43
9.2 Model name	43
9.3 Use kernels	43
9.4 Feature sets	43
9.5 Model Type	43
9.5.1 Classification	43
9.5.2 Regression	44
9.6 Cross-validation type	45
9.6.1 Leave one subject out	45
9.6.2 Leave one subject per group out	45
9.6.3 Leave one block out	45
9.6.4 Leave one run/session out	45
9.6.5 Custom	45
9.7 Data operations	46
9.7.1 Mean centre features	46
9.7.2 Other Operations	46
10 Run model	47
10.1 Load PRT.mat	47
10.2 Model name	47
 III Data processing examples	 49
11 Data set 1	51
12 Data set 2	53
 IV Advanced topics	 55
13 PRT structure	57
14 List of PProNTTo functions	61
14.1 prt.m	61
14.2 prt_apply_operation.m	61
14.3 prt_check_design.m	62
14.4 prt_compute_weights.m	62
14.5 prt_cv_model.m	62
14.6 prt_cv_opt_param.m	63
14.7 prt_data_conditions.m	63
14.8 prt_data_modality.m	64
14.9 prt_data_review.m	64
14.10prt_defaults.m	65
14.11prt_fs.m	65
14.12prt_func2html.m	66
14.13prt_get_defaults.m	66
14.14prt_get_filename.m	66
14.15prt_init_fs.m	66
14.16prt_init_model.m	67
14.17prt_latex.m	68

14.18prt_load.m	68
14.19prt_load_blocks.m	69
14.20prt_model.m	69
14.21prt_normalise_kernel.m	70
14.22prt_permutation.m	70
14.23prt_preproc.m	70
14.24prt_remove_confounds.m	71
14.25prt_stats.m	71
14.26prt_struct2latex.m	71
14.27prt_text_input.m	71
14.28prt_ui_compute_weights.m	72
14.29prt_ui_cv_model.m	72
14.30prt_ui_design.m	73
14.31prt_ui_kernel_construction.m	73
14.32prt_ui_main.m	74
14.33prt_ui_model.m	74
14.34prt_ui_prepare_data.m	75
14.35prt_ui_prepare_datamod.m	75
14.36prt_ui_results.m	76
14.37prt_ui_reviewCV.m	76
14.38prt_ui_reviewmodel.m	77
14.39prt_ui_select_class.m	77
14.40prt_ui_select_reg.m	78
14.41machines	78
14.41.1 machines\prt_KRR.m	78
14.41.2 machines\prt_machine.m	78
14.41.3 machines\prt_machine_RT_bin.m	79
14.41.4 machines\prt_machine_gpml.m	79
14.41.5 machines\prt_machine_krr.m	80
14.41.6 machines\prt_machine_rvr.m	80
14.41.7 machines\prt_machine_svm_bin.m	81
14.41.8 machines\prt_rvr.m	81
14.41.9 machines\prt_weights.m	82
14.41.10 machines\prt_weights_bin_linkkernel.m	83
14.41.11 machines\prt_weights_svm_bin.m	83
14.42utils	83
14.42.1 utils\prt_centre_kernel.m	83
14.42.2 utils\prt_checkAlphaNumUnder.m	83
14.42.3 utils\prt_normalise_kernel.m	83

V Bibliography

Chapter 1

Introduction

This is where we introduce PRoNTo and explain what we wanted to do...
Cite a few important references like [1], or [2], or another [3] and obvious [4].

1.1 Goals

This comes from the Harvest application document.

Advances in neuroimaging techniques have radically changed the way neuroscientists address questions about functional anatomy, especially in relation to behavioural and clinical disorders. Many questions about brain function, previously investigated using electrophysiological recordings in animals can now be addressed non-invasively in humans. Such studies have yielded important results in cognitive neuroscience and neuropsychology. Amongst the various neuroimaging modalities available, Magnetic Resonance Imaging (MRI) has become widely used due to its relatively high spatial and temporal resolution, and because it is safe and non-invasive. By selecting specific MRI sequence parameters, different MR signals can be obtained from different tissue types, giving images with high contrast among organs, between normal and abnormal tissues and/or between activated and deactivated brain areas. MRI is often sub-categorized into structural MRI (sMRI) and functional MRI (fMRI). Structural and functional MRI data are inherently multivariate in nature, since each scan contains information about tissue structure, or brain activation, at thousands of measured locations (voxels). Considering that most brain functions are distributed processes involving a network of brain regions, it would seem desirable to use the spatially distributed information contained in the data to give a better understanding of brain functions in normal and abnormal conditions.

The typical analysis pipeline in neuroimaging is strongly rooted in a mass-univariate statistical approach, which assumes that activity in one brain region occurs independently from activity in other regions. Although this has yielded great insights over the years, and continues to be the tool of choice for data analysis, there is a growing recognition that the spatial dependencies among signal from different brain regions should be properly modeled. The effect of interest can be subtle and spatially distributed over the brain - a case of high-dimensional, multivariate data modeling for which conventional tools may lack sensitivity.

Therefore, there has been an increasing interest in investigating this spatially distributed information using multivariate pattern recognition methods. Where pattern recognition has been used in neuroimaging, it has led to fundamental advances in the understanding of how the brain represents information and has been applied to many diagnostic applications. For the latter, this approach can be used to predict the status of the patient scanned (healthy vs. diseased or disease A vs. B) and can provide the discriminating pattern leading to this classification. Several active areas of research in machine learning are crucially important for the difficult problem of neuroimaging data analysis: modelling of high-dimensional multivariate time series, sparsity, regularisation, dimensionality reduction, causal modeling, and ensembling to name a few. However, the application of pattern recognition approaches to the analysis of neuroimaging data is limited mainly by the lack of user-friendly and comprehensive tools available to the fundamental, cognitive, and clinical neuroscience communities. Furthermore, it is not uncommon for these

methods to be used incorrectly, with the most typical case being improper separation of training and testing datasets.

The aim of the project is to develop a toolbox based on machine learning techniques for the analysis of neuroimaging data (PRoNTo: Pattern Recognition Neuroimaging Toolbox). The toolbox will be a MATLABbased software package specifically designed for the analysis of brain imaging data including steps for feature extraction according to the experimental design, feature selection, machine training and testing. Initially it will focus on supervised learning approaches such as Support Vector Machines, Gaussian Processes, etc. Other techniques will be added once a consistent framework and data flow has been implemented.

The toolbox will facilitate the interaction between machine learning and neuroimaging communities. On one hand the machine learning community will be able to contribute to the toolbox with novel published machine learning algorithms. On the other hand the toolbox will provide a variety of tools for the neuroscience and clinical neuroscience communities, enabling them to ask new questions that cannot be easily investigated using existing statistical analysis tools. The toolbox code will be distributed for free, but as copyright software under the terms of the GNU General Public License as published by the Free Software Foundation.

1.2 Main contributors

PRoNTo is developed by the Machine Learning & Neuroimaging Laboratory, Computer Science department, University College London, UK (<http://www.mlnl.cs.ucl.ac.uk>) and associated researchers.

The main contributors, in alphabetical order, are:

Dr. John Ashburner is a reader at the Wellcome Trust Centre for Neuroimaging at the UCL Institute of Neurology. He is mainly interested in modeling brain anatomy from MR scans, and more recently in applying pattern recognition methods to make predictions about individual subjects. He is a co-developer of the SPM software (intra- and inter-subject registration, tissue classification, visualization and image file formats), which is used internationally by thousands of neuroimaging researchers. He has authored or co-authored 90 papers in international journals (h-index of 50) and written a number of book chapters;

Dr. Carlton Chu is a research fellow in brain imaging at the National Institute of Mental Health (NIMH), NIH. He received the B.Eng. degree (1st class Honours) from Auckland University, New Zealand, in 2002 and the master of Biomedical Engineering from University of New South Wales, Australia, in 2004. Carlton obtained a PhD in Neuroimaging method from University College London in 2009, working in the statistical methods group at the prestigious Wellcome Trust Centre for Neuroimaging, creators of the famous SPM program. There he developed innovative new pattern recognition methods to automatically detect the early stages of neurodegenerative diseases such as Alzheimers and Huntingdons just from structural brain images. In 2007, Carlton won the first prize in the 2nd Pittsburgh Brain Activity Interpretation Competition (PBAIC), a prestigious international competition involving the application of machine learning to the problem of classification of brain activity. He led a small research team to victory, acclaim from peers in the field, and the \$10K first prize. His current research interests include brain state decoding, neurodegenerative disease classification, and applying pattern recognition method to study brain networks;

Dr. Andre Marquand is a Post-Doctoral Research Fellow at the Centre for Neuroimaging Sciences, Kings College London and an Honorary Post-Doctoral Research Fellow at the Centre for Computational Statistics and Machine Learning at University College London. His research focuses on the application of probabilistic machine learning techniques to neuroimaging data, particularly for clinical applications. His recent work includes the application of multi-class and multi-modality pattern classification methods to neuroimaging and in particular to detecting the effects of psychotropic medication on patterns of brain activity;

Dr. Janaina Mourao-Miranda is a Wellcome Trust Senior Research Fellow at Centre for Computational Statistics and Machine Learning (CSML), UCL and at the Centre for Neu-

roimaging Sciences (CNS), KCL. Her research focuses on developing and applying pattern recognition methods to analyze neuroimaging data, in particular brain activation and structural patterns that distinguish between controls and patients. Recent work includes the development and application of spatio-temporal SVM, one-class SVM to detect patients as outliers and in-depth studies of kernel methods for brain decoding;

Dr. Christophe Phillips is FRS-FNRS Research Associate at the Cyclotron Research Centre and adjunct Assistant Professor at the Department of Electrical Engineering and Computer Science, University of Lige, Belgium. His research focuses on the processing of multi-modal neuroimaging data. Recent work within the field of brain decoding aimed at distinguishing between levels of consciousness in unresponsive patients or between typical and atypical Parkinson Disease patients using Positron Emission Tomography (PET) imaging, as well as tracking mnesic traces in trained healthy subjects with fMRI;

Dr. Jonas Richiardi is a post-doctoral research fellow, jointly affiliated to the EPFL engineering school (Medical Image Processing Laboratory) and the Geneva university hospital (Department of Radiology and Medical Informatics). His research interests include brain connectivity and resting-state networks analysis, interpretability of brain decoding results, functional biomarkers, learning with graphs, and machine learning for neuroimaging. He was co-chair of the 2010 Brain Decoding Workshop at the Int. Conf. on Pattern Recognition and a program chair of the Pattern Recognition in NeuroImaging workshop 2011;

Dr. Jane Rondina is a Wellcome Trust Post Doctoral Research Associate at Centre for Neuroimaging Sciences (CNS), KCL and researcher as an honorary member at Centre for Computational Statistics and Machine Learning (CSML), UCL. Her current work includes the development of a feature selection method for classification in neuroimaging and analysis of features stability. Her research interests also include the development of pattern recognition methods using data from different modalities / measures in neuroimaging;

Ms Maria J. Rosa is a PhD student at the Wellcome Trust Centre for Neuroimaging, Institute of Neurology, University College London, UK. Her areas of interest include Bayesian model selection methods for fMRI and Dynamic Causal Modelling, EEG-fMRI fusion and biophysical models of neurovascular coupling;

Ms Jessica Schrouff has a Master in Biomedical Engineering and is currently pursuing a PhD in neuroimaging at the Cyclotron Research Centre, University of Lige, Belgium, under the supervision of Dr C. Phillips. Her project focuses on the tracking of mnesic traces of learned images in trained healthy subjects with fMRI and EEG data, as well as the classification of patients from PET images.

1.3 Acknowledgements

PRoNTo v1 (2011) is the deliverable of a Pascal Harvest project coordinated by Dr. J. Mourao-Miranda and its development was possible with the financial and logistic support of

- the Department of Computer Science, University College London (<http://www.cs.ucl.ac.uk>);
- the Wellcome Trust (<http://www.wellcome.ac.uk/>);
- PASCAL2 (<http://www.pascal-network.org>) and its HARVEST programme;
- the Fonds de la Recherche Scientifique-FNRS (<http://www.fnrs.be>), Belgium;
- Swiss National Science Foundation (PP00P2-123438) and Center for Biomedical Imaging (CIBM) of the EPFL and Universities and Hospitals of Lausanne and Geneva.

PRoNTo is written for MATLAB X.Y (R20ZZb) and onwards. Some routine may need to be compiled for your specific OS.

Part I

Graphic User Interface

Chapter 2

Data & design

2.1 Introduction

The first step in a statistical analysis of neuroimaging data, whether it's in a pattern recognition or general linear model framework, usually entails providing to the analysis software all the information regarding the data and experimental design. P_{RoNT}o is no exception. After pre-processing the data (if required) using another software, the analysis in P_{RoNT}o starts with the 'Data and Design' module. In this module the user can enter the image/scan files, experimental conditions (TR, durations and onsets of events), as well as other parameters, covariates and regression values. P_{RoNT}o supports multi-modality datasets and therefore it allows the user to enter more than one data modality, such fMRI, MRI, PET and ASL, per analysis. This module is therefore essential for the rest of the framework and stores all the information that is needed from the data to be used by the rest of the software modules, such as data preparation, model specification and estimation.

Below is a summary of what the 'Data and Design' module does. The Methods section discusses how the module is organised and what its main output is. It also mentions a few issues that need to be taken into consideration when entering the information and how they affect subsequent steps. This chapter then presents the graphical user interface (GUI) that is used to enter the data and design information and how it is used. Finally, the chapter finishes by mentioning the corresponding 'Data and Design' Matlab batch module, and particular issues that do not apply to the GUI.

2.2 Methods

P_{RoNT}o provides two types of interfaces for entering the data and design information, a P_{RoNT}o-specific graphical user interface (GUI) and the Matlab batch system that is also currently used by SPM. These two interfaces are also available for the other modules, as discussed in the Introduction chapter. The 'Data and Design' GUI can be launched by typing 'prt' in the Matlab window and then clicking the first button on the left, in the main steps panel. The 'Data and Design' module in the Matlab batch is called either by first typing 'prt' and clicking the 'Batch' button or by typing 'prt_batch'. The user can then find on top of the batch a P_{RoNT}o menu and under this menu the first module corresponds to the data and design module.

The information that needs to be entered is almost exactly the same for both the GUI and batch (the small differences are explained below in the Matlab batch section) and, more importantly, the output is exactly the same. Therefore it is up to the user to decide which system is best suited for his/her analyses. For instance, the GUI can be used as a first approach to the toolbox and by users not familiar with SPM, whilst the batch can be used by more advanced or SPM users, who know how to take advantage of the batch system to optimise their analyses.

The output of the 'Data and Design' module is the PRT structure (as discussed in the Introduction). This structure contains subfields with all the information that is needed from the data for the subsequent analysis steps and it is saved in a 'PRT.mat' file. For power users the fields of this structure can be edited directly and saved, therefore bypassing the need to use the GUI or

Matlab batch to create the PRT. However, this structure is the core of PRoNTTo and should be carefully created because it affects everything else.

Regardless of which interface the user chooses to enter the data and design (GUI or batch), the organisation is very similar and starts (after choosing the directory to save PRT.mat) with the definition of Groups. In neuroimaging datasets, it is common to have a few subjects with a lot of images/scans per subject, such as the time-series in fMRI. However, the opposite is also common: lots of subjects with one image per subject, such as encountered in PET or MRI studies. Therefore, for each group, PRoNTTo provides two ways of entering the rest of the information, i.e. subjects, modalities and design, which are referred to as the ‘select by subject’ or ‘select by scans’ option, respectively (as is shown below). If one chooses to enter the data by ‘scans’, PRoNTTo allows the user to enter, for each modality, all subjects (one image/scan per subject) at once, which is a lot quicker than entering each subject at a time. This option however is not appropriate for modalities which have an experimental design and more than one image per subject, such as fMRI. For these datasets the user should choose the ‘subjects’ option. For each subject one can specify the modalities, experimental conditions and enter more than one image/scan. Both options are valid and produce exactly the same PRT.mat (if used with the same dataset).

The ‘Data and Design’ module also allows the user to review the information that has been entered (through the GUI, batch or manually). The main aim of the ‘Review’ function is to check if the data and design has been correctly specified. It can also be used to inspect if the design is appropriate for subsequent analysis. For example, the review window shows the number of subjects in each group, and for modalities with experimental design, it can be used to show and alter the number of selected and discarded scans. For datasets such as fMRI, the discarded scans is a very important issue and needs to be carefully addressed when specifying the data and design. As is well known, the hemodynamic response function (HRF) is a delayed and dispersed version of the underlying neuronal response to an experimental event. This means that, depending on the TR, the effect of the HRF can be felt over multiple scans, and therefore the scans are not independent and might contain information from both past and present events. This can confound subsequent pattern recognition analyses and needs to be accounted for, by discarding the overlapping scans. In the Review module one can opt to discard more or less scans by providing two parameters that determine the shape of the HRF: its delay (peak time) and overlap (dispersion). When using the GUI, the default value for these two parameters is 6 seconds for datasets with design (0 seconds for datasets with no design) and can only be changed in the ‘Review’ window (as shown below). In the Matlab batch, the default value for these parameters is also 6 seconds but can be changed directly within the batch. It should be changed to 0 for other modalities other than fMRI.

The steps to specify the information relative to the data and design using both the GUI and the Matlab batch system are described in the following sections.

2.3 Graphical User interface

The graphical user interface to specify the data and design is presented in Figure 2.1. As mentioned above, this GUI can be launched from PRoNTTo’s main menu (first button on main steps panel).

2.3.1 PRT directory

The first thing the user should specify is the directory in which to save the PRT structure. This can be done by browsing existing directories (previously created by the user) from the top of the data and design interface (Figure 2.1). It is recommended to have different directories for different datasets (not modalities) because PRoNTTo overwrites an existing PRT in the selected directory. The later modules in PRoNTTo will then add more fields to this structure with further information, such as the models, features and kernels used in subsequent analyses. The file created is called ‘PRT.mat’.

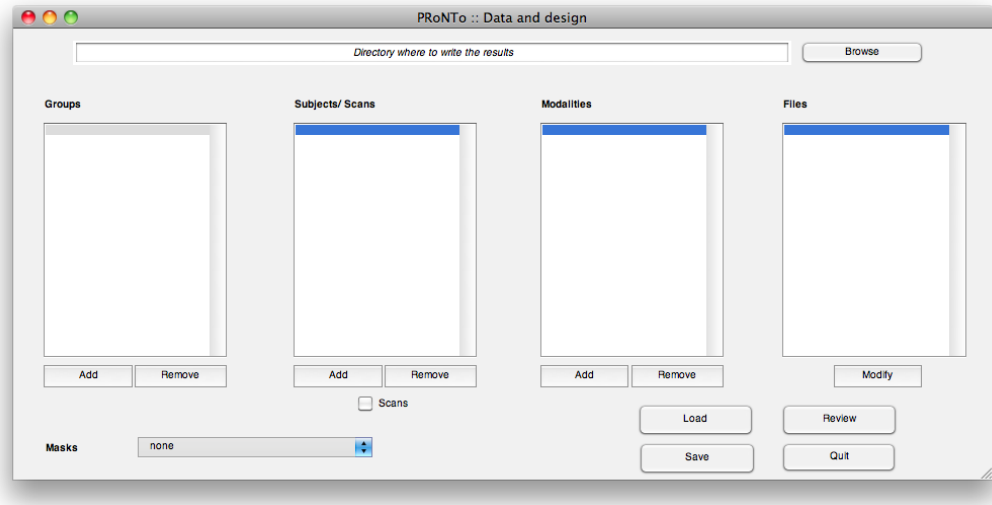


Figure 2.1: Data and design graphical user interface. This interface allows the user to enter all the information relative to the data, including the experimental design and masks. After introducing all the fields, PRoNTTo creates the PRT structure, which is saved in the specified directory, as ‘PRT.mat’ file.

2.3.2 Groups

The group panel allows one to add or remove a group of subjects. The minimum number of groups is one, but there is no maximum number. When ‘Add’ is clicked, the user should provide a name to the group. Any alphanumeric string is sufficient. When ‘Remove’ is clicked, all the information relative to this group (including all subjects and corresponding data) is deleted. PRoNTTo does not restore the deleted information and it can only be re-entered again by clicking ‘Add’.

The following panel after ‘Groups’ is ‘Subjects/Scans’. Here, as mentioned above, there are two ways of entering the data: by ‘subjects’ or by ‘scans’. The former is chosen by clicking ‘Add’ under the ‘Subjects/Scans’ panel and filling in the fields for each added subject at a time. The latter is done by clicking the tick box ‘Scans’ under the ‘Subjects/Scans’ panel. The subjects panel is then de-activated and the user can enter the modalities and files straight away. The fields to be filled under these two options are described below.

2.3.3 Subjects

Select by scans The ‘select by scans’ option allows the users to skip the subject step. To identify that this option has been selected, PRoNTTo writes ‘scans’ in the subjects panel (Figure 2.2). The user can then add modalities and for each modality a new window will appear (bottom of Figure 2.2). It is important to remember that when the ‘scans’ box is clicked all the information in the subjects panel is automatically deleted! Unselecting the ‘scans’ box also deletes all the information!

Select by subjects The ‘Subjects/Scans’ panel allows the user to add/remove subjects. This panel works exactly like the groups panel, but the subject name is automatically generated. For each subject one can then specify the modalities in the next panel.

2.3.4 Modalities

The modalities panel works like the group and subjects panel, but allows one to add and remove modalities. When a modality is added, a name needs to be provided (unless the modality has already been defined for a previous subject or through the masks menu, see below). The following steps are slightly different if one ticks the ‘scans’ box or not.

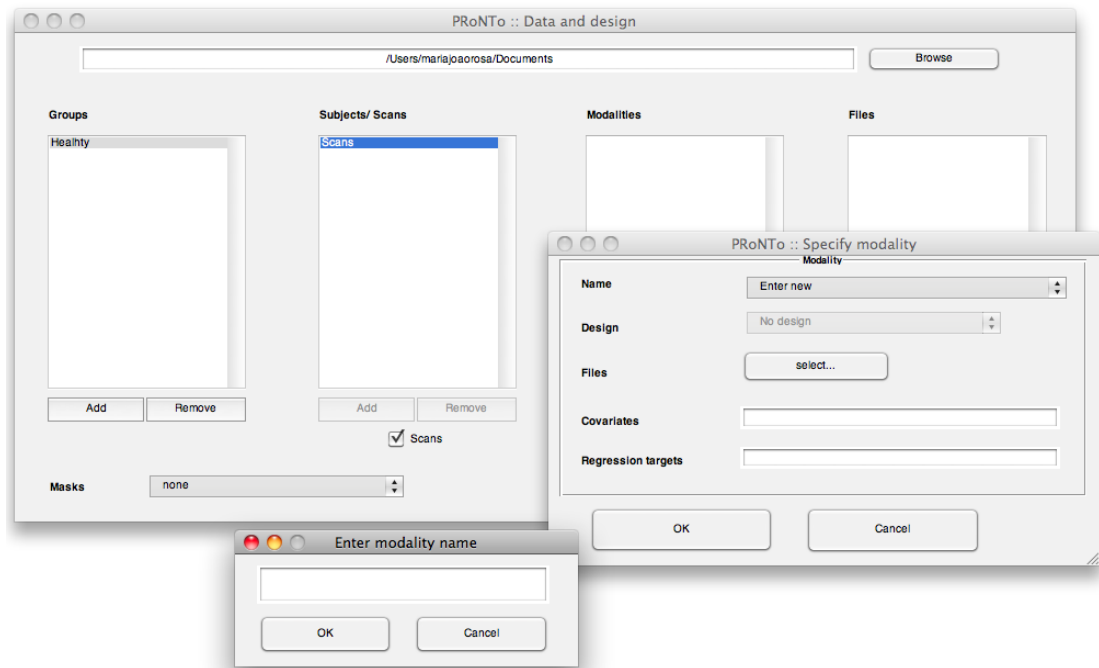


Figure 2.2: Data and design graphical user interface. If one chooses to specify everything using the ‘Scans’ option (tick box below the ‘Subjects/Scans’ panel), one can introduce the data for all subjects at once for each modality, but one cannot specify any design. This is the optimised approach when one has a lot of subjects with only one image/scan per subject, such can be the case of MRI and PET datasets.

Select by scans Here the data is assumed to have been acquired without an experimental design, and therefore the ‘No design’ option is automatically selected and cannot be changed (bottom window in Figure 2.2). However, in select by scans, the user can also introduce ‘Covariates’, i.e. a variable that covaries with the data (subjects) but of no interest to the subsequent analyses. This option is not yet functional in version 1.0 of PRoNTTo! The last empty field can be used to enter ‘Regression targets’ (Figure 2.2). This option allows the users to introduce a real number per subject to be used later for regression if that is the case.

Select by subjects When entering the data by subjects, the modality window allows one to specify the experimental design (Figure 2.3). Here there are three options. The last option is simply ‘No design’, which means that for this modality there are no experimental conditions. The first option is to load an SPM.mat with a previously specified design. This option can be chosen if the user has created an SPM structure containing all the experimental fields using the SPM software. In this case, the user does not need to specify anything else, only the files (scans/images) for this subject/modality. The design information is extracted directly from the SPM structure and saved in PRT.mat. Finally, the ‘Specify design’ option allows one to introduce all the conditions (durations and onsets), TR and other parameters corresponding to the experimental paradigm used for this subject and modality.

Design To create a new design one selects the option ‘Specify design’ as explained in the previous paragraph (Figure 2.3). This will then open another window (after choosing how many conditions you have) (Figure 2.4). In this window one can then write the names, onsets, and durations of each condition. The units in which this information is read is specified below. There are two options ‘Scans’ or ‘Seconds’. If the unit scans is selected, it is good to bear in mind that PRoNTTo follows the convention, adopted in SPM, that the first scan is scan 0. In the durations field, one can introduce as many values as the number of onsets or just simply one value, which assumes the events all have the same duration. In this window there is also the

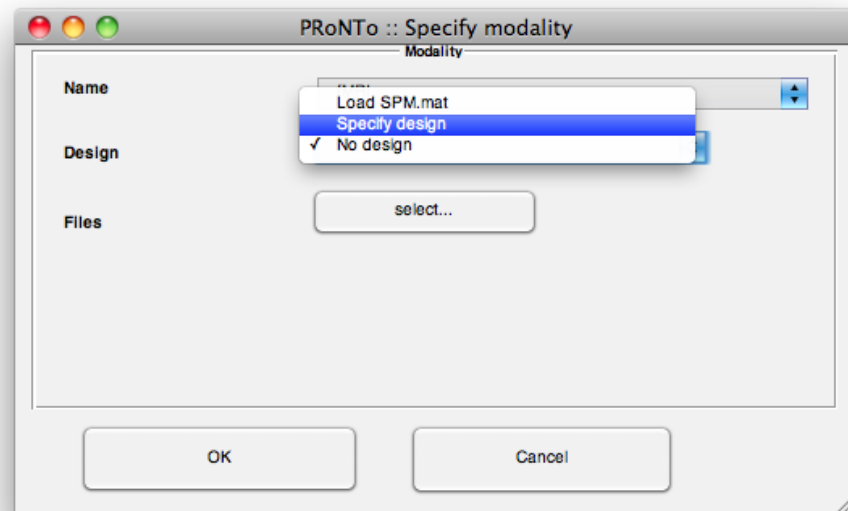


Figure 2.3: Data and design graphical user interface. The design menu in the modality window (when one uses the select by subject option) allows one to load a previously specified design from an SPM.mat file, create a new design or simply select no design, which usually applies to modalities where there is no experimental task, such as MRI or PET.

option of introducing the Interscan Interval (TR), which is always read in seconds. Finally, there is also an option (which is again not yet functional in version 1.0 of PRoNTTo!) to introduce covariates, which, in this case, correspond to any variable that varies along with the experimental events but of no interest for further analyses.

Files Finally, independent of the way the user entered the information (by subjects or scans) the ‘Files’ option allows one to choose which image files to use (Figure 2.5). This will open another window that shows all image files available in each directory. These can be selected one by one or all at once, by using the mouse’s right button on the right panel of the window.

All that is needed for each group, subject and modality has been specified and can now be viewed on the main window (Figure 2.6) under each panel. The last panel shows which files have been entered for each modality and can be modified directly (click Modify). When Modify is clicked and no files are then selected all the previous files are deleted! Figure 2.6 shows how the data and design interface should look like once all the fields have been specified (using select by subject).

2.3.5 Masks

This popdown menu on the bottom of the main data and design window is where the user enters a binary image mask for each modality. This mask can be previously created by the user or simply chosen from a list of default masks available in the masks directory of PRoNTTo. Every modality has to have a mask, which can be the same for all modalities. This is a first-level mask and is used simply to optimise the prepare data step by discarding all uninteresting features, such as voxels outside the brain. Later in the analysis one can choose another mask (second-level mask) that is more relevant to the scientific question and that can, for example, restrict the analysis to certain areas of the brain. To specify the mask one needs only to select the modality and then enter an image file. If the modalities have not yet been created, then one can create the modalities here, which will then appear in the modality panel.

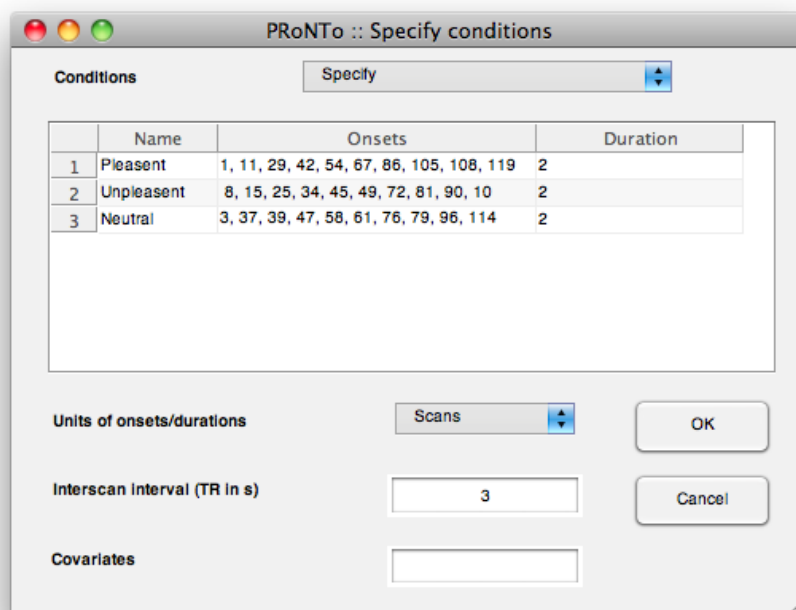


Figure 2.4: Data and design graphical user interface. The ‘specify conditions’ window is available from the modality interface when the user chooses to enter the data by subjects and clicks ‘specify design’. This window is used to enter the conditions (names, onsets and durations) as well as the units of design, TR and covariates.

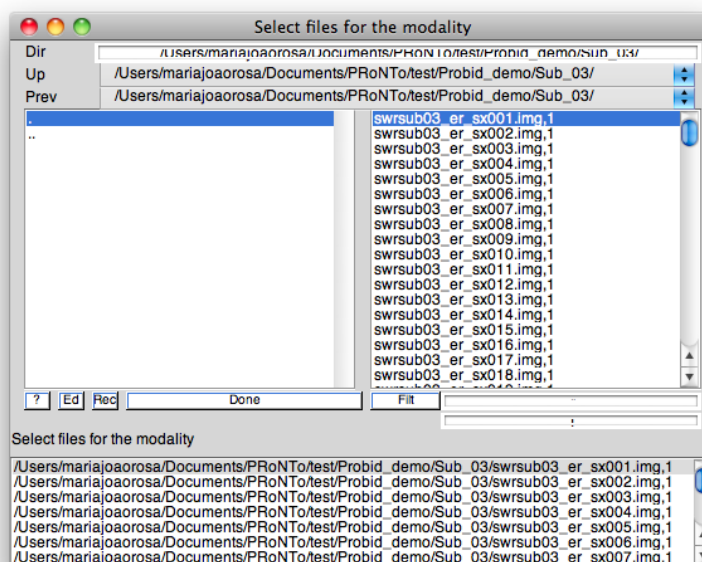


Figure 2.5: This window is called when one clicks ‘Files’ and is used to select the scans/images for each subject/modality.

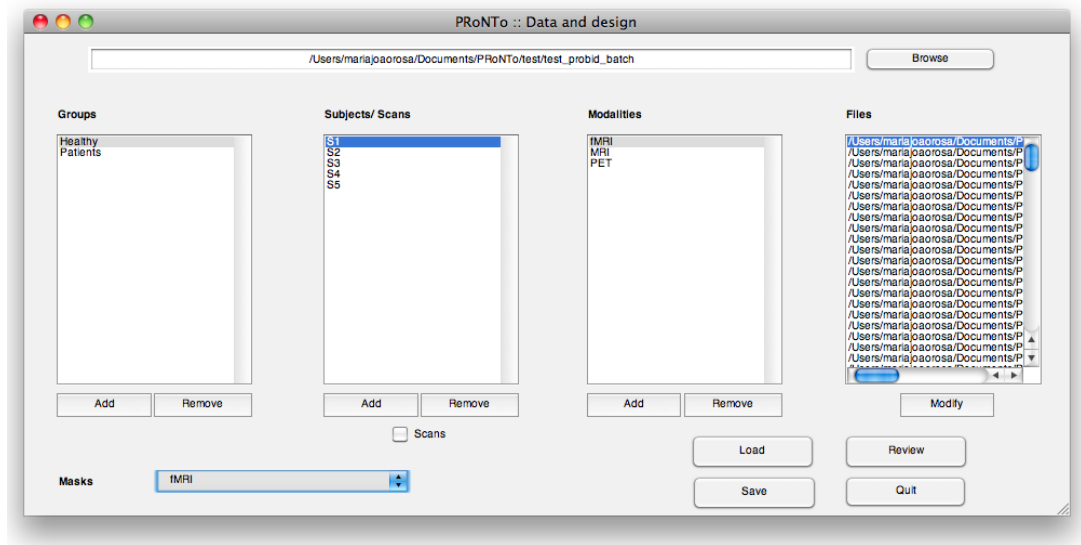


Figure 2.6: Data and design graphical user interface. After filling in all the fields using the select by subject option (the select by scans case is very similar) the data and design interface should look like this example figure.

2.3.6 Review

The ‘Review’ button allows one to review the data and design for each modality (Figure 2.7). On the top right is the information relative to the number of groups and modalities that have been entered. The plot on the left displays the number of subjects per group. This is particularly important to check if the design is too unbalanced in terms of subjects. Then on the bottom right panel is the design information for each modality (if the selected modalities have an experimental design). Here, the user can view the number of conditions and can also edit the parameters that control the HRF delay and overlap (as explained above). The user can change the default value of 6 seconds and the effect is immediately seen on the number of scans plotted on the left (number of selected scans and number of discarded scans for each condition). The higher the value of the HRF peak and overlap, the higher the number of discarded scans. One can also read on the main Matlab window information regarding which group/subjects have had some scans discarded. The information below the HRF parameters corresponds to the interval between successive scans before and after the HRF delay/overlap correction. These values also change according to the changes entered in the boxes above.

2.3.7 Load, Save and Quit

The ‘Save’ button allows the user to create the PRT.mat file with the PRT structure containing all the information that has been specified here (Figure 2.6). Incomplete information cannot be saved. At least one group should have all the required fields so that PRT.mat can be created. ‘Load’ allows the user to load the data and design information from a previously saved PRT.mat. The user can then edit the fields and update PRT by clicking again the ‘Save’ button. It’s very important to click ‘Save’ because all the other steps in the analysis rely on the PRT structure. Without this structure one cannot proceed. However, when the PRT.mat contains fields that have been added by the ‘Prepare data’ or other modules, if the Save button is clicked, these fields will be deleted. The option ‘Quit’ allows the user to leave the interface without saving the information. This is also the case when the user closes the window without first using the Save button.



Figure 2.7: Data and design graphical user interface - ‘Review’ window. This window allows the user to check the data and design, including the number of subjects per group. It also allows the user to change the HRF delay and overlap parameters that control the number of discarded scans (appropriate only for modalities such as fMRI). When there is no experimental design only the top plot and information is shown.

2.4 Matlab batch interface

The options presented in the ‘Data and Design’ GUI, mentioned above, are all available in the Matlab batch interface (Figure 2.8). However, there are a few things in the batch that differ from the GUI. One issue to note here is that, when using the batch one needs to be very careful with the names of the modalities specified for each subject (or using select by scans) and specified for each mask. The number of modalities should be exactly the same for each group and subject and the names should be consistent between groups/subjects and correspond to the names of the modalities under the masks field. In the GUI the names are made automatically consistent. The names of the conditions should also be the same across subjects and will be later used to define classes in the ‘Specify model’ batch module.

Another issue is the HRF delay and overlap correction values. In the batch, the user can directly alter these values (instead of having to use the ‘Review’ window) but the default is 6 seconds and should be changed to 0 seconds for other modalities other than fMRI.

As mentioned in the Introduction, the batch job can be saved as a .mat, and loaded again whenever needed, or as a .m that can be edited using the Matlab editor. This is a powerful tool that can make the specification of the data and design a lot easier and quicker, for example by editing and scripting existing batch files (for further information see the Matlab batch chapter below).

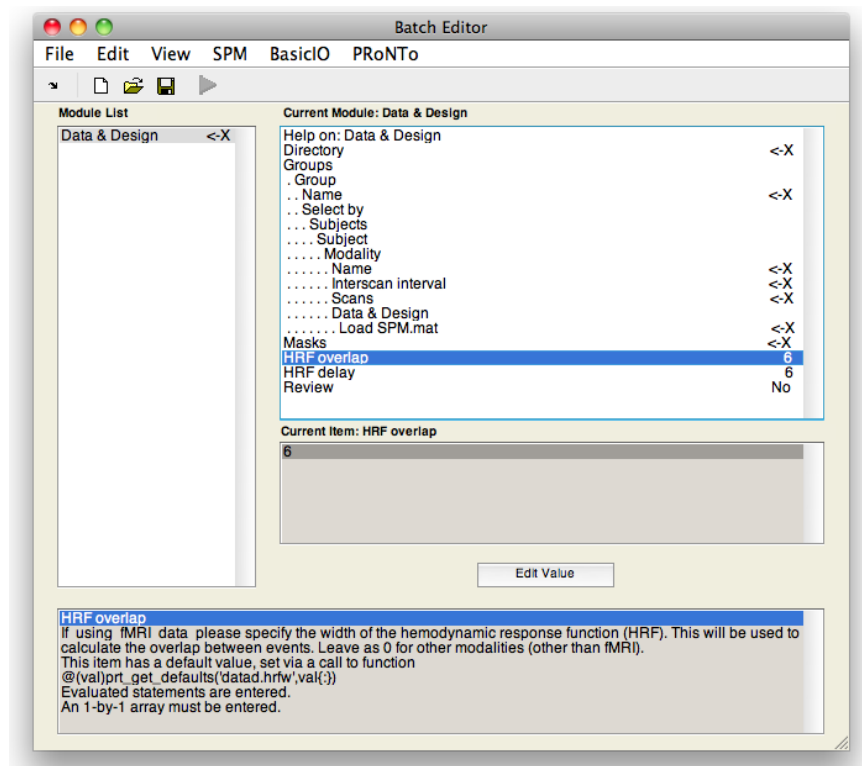


Figure 2.8: Data and design module in Matlab batch. The Matlab batch contains two extra options relative to the Data and Design interface. These options allow one to specify the delay and overlap of the HRF response (in the GUI it can only be changed in the ‘Review’ window), and which are then used to determine the number of discard scans.

Chapter 3

Prepare feature set

3.1 Introduction

One of the main inputs of a machine learning algorithm consists in a $N_{samples} \times N_{features}$ data matrix, containing the values of selected features for each sample. This matrix can either be input directly into the machine or be used to compute a similarity matrix, or kernel, of the size $N_{samples} \times N_{samples}$, which is then input into the classification/regression algorithm [kernel trick, ref].

The "Prepare Feature Set" step computes both the feature and similarity matrices from one or more modalities, as defined in the previously built dataset (see chapter 2). It allows detrending the features in the case of time series (such as fMRI) and scaling each image by a constant factor (input by the user) in the case of quantitative modalities (such as PET). Masks can be specified to perform the classification/regression on specific voxels only (e.g. Regions of Interest).

Please note that only modalities containing the same number of features (i.e. selected voxels) can be included in the same FS. This will be typically the case when more than one run was acquired for each subject, the different runs being entered as different 'modalities' in the dataset building (e.g. modality 1 is 'fMRI_run1', modality 2 is 'fMRI_run2',...). In all other cases, a feature set has to be computed for each modality.

3.2 Methods and resources

After the selection of the dataset and of which modality to include in the FS, the toolbox accesses each image, i.e. it gets the value of the voxels which are comprised in the first level mask selected for that modality (mask specified at the data and design step, see chapter "Data and Design"). This access is performed by 'blocks' of features, not to overload the RAM memory. In the case of time-series, the user can specify detrending methods and parameters to apply to the time course of each feature. Methods comprise a polynomial detrending (parameter: order of the polynomial) or a Discrete Cosine Transform high-pass filter (SPM, ref, parameter: frequency cutoff in seconds). An example of a linear detrending (polynomial detrending of order 1) was represented in Fig. `reffig:lindetrend`.

For each modality, the (detrended) features are then written in a file array (SPM, ref, with a '.dat' extension), on the hard drive (in the same directory as the dataset). Please note that in the case of large datasets, this operation may require many Gb of free space on the hard drive and long computational times. Therefore, if the first condition can't be fulfilled, we recommend the use of external drives for the whole analysis. Regarding the computational expenses, we tried to minimize their effect by computing the features only once per modality: when preparing other feature sets using the same modality and detrending parameters, the same file array will be accessed.

Be careful that using the same modality but different detrending methods and/or parameters will force the re-computation of the file array for the considered modality. In the same way, changing the dataset (`PRT.mat`) from directory might lead to the re-computation of the feature sets if the file arrays were not moved accordingly.

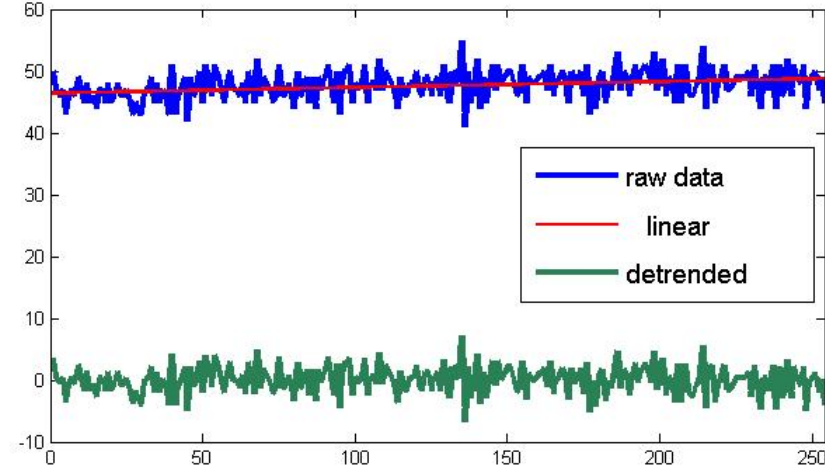


Figure 3.1: Example of detrending: the original signal over time of one feature (in blue) was approximated by a polynomial of order 1 (red line), which was then subtracted from the original signal to give the detrended signal (in green).

From the feature set(s), the kernel (similarity matrix) can then be computed. Different options can be specified:

- All scans/ All conds: In all scans the similarity will be computed between all scans within the time series of all subjects and in the all conds the similarity is computed only between the scans corresponding to the specified conditions of interest (see "Data and Design"). By default, the toolbox will use all scans to compute the kernel. With large datasets however, computational expenses can be reduced by selecting the last option. We would therefore recommend this last option for cases similar to multi-subject fMRI studies with designed stimulation.
- Additional mask for selected modality: this option allows the specification of a 'second-level' mask, which would for example define Regions of Interest (ROIs) on which the classification/regression can be performed. In this case, the voxels used to compute the kernel (and only the kernel) would be the ones contained in both the first and second levels masks. Therefore, using one first-level mask and two second-level masks would create two kernels but one file array.
- Normalisation (has to be called scaling): allows the specification of constant values to scale each scan. The user has to enter a .mat containing a variable called 'scaling' and of the same size as the number of scans in that modality. In case of quantitative modalities such as PET, this step is required since it insures the convergence of the machine learning algorithm.

These three options are performed at the kernel level only. This means that any change in one of these options would lead to the computation of a new kernel but not to the (re)computation of the file arrays. The use of different second-level masks or scaling parameters can therefore be easily envisaged.

The `PRT.mat` structure saves all information linked to the file arrays in a `fas` field (standing for "File Array Structure"), which size corresponds to the number of selected modality in all feature sets. The selected options and link to the kernel are stored in a `fs` field (standing for "Feature Set"), which size corresponds to the number of feature sets defined by the user.

3.3 Graphical User interfaces

After clicking on the “Prepare Feature Set” button in the main interface (see Fig. 3.2), a second window will appear, allowing the user to select a dataset (Fig. 3.3.A), to name the FS (Fig. 3.3.B) and to define the number of modalities which should be included in the FS (Fig. 3.3.C), see section 3.1 for a comment on this last point).

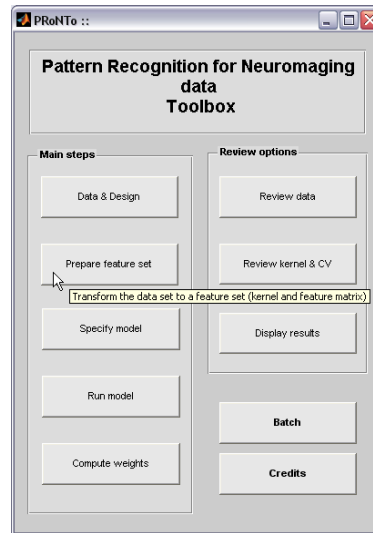


Figure 3.2: Main interface: button to launch the 'Prepare Feature Set' step.

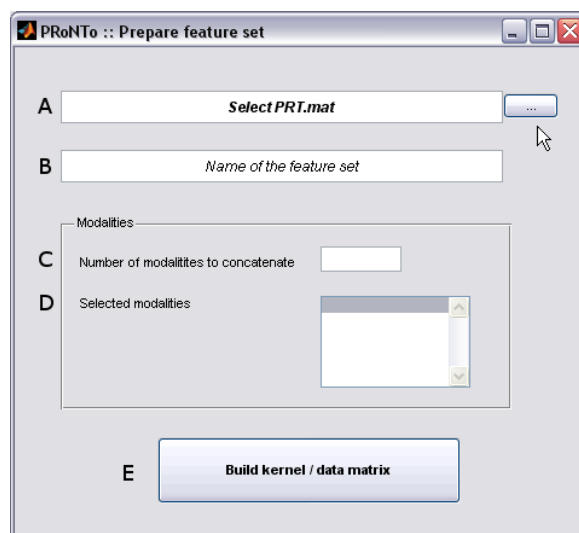


Figure 3.3: Interface of the 'Prepare Feature Set' step: A. Dataset selection: type the full name (with path) or browse to select the dataset to prepare. B. Type the FS name, which will be used to save the kernel as a .mat on the hard drive. C. Number of modalities to select. D. List containing the names of the modalities included in the FS (no user interaction possible). E. Click to build the feature set and kernel.

To define the number of modalities to include, the user should click in the appropriate box (Fig. 3.3.C), type the number and then 'return' (arrow for return?). This will launch a third window, allowing the specification of the different options and parameters for each modality (Fig. 3.4). When the dataset contains only one modality, this window is launched automatically.

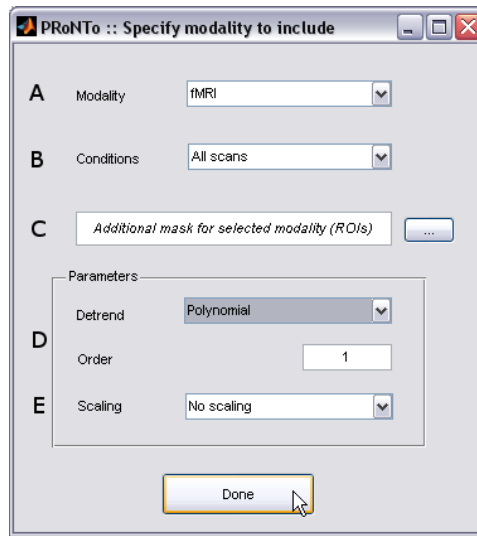


Figure 3.4: Specification of options and parameters for each modality: A. Select the modality name from a pull-down menu. B. All scans/All conds. C. Second-level mask selection. D. Detrend and its parameter. E. Scaling of the scans or not.

In this third window, the user has to choose which modality to include based on its name (Fig. 3.4.A) and which scans to use to build the kernel (all or only those linked to the design, Fig. 3.4.B). All other options are facultative. They include:

- the specification of a second-level mask (Fig. 3.4.C): type the full name (with path) of the mask or browse to select the mask image. When left empty or untouched, voxels are selected from the first-level mask specified in the data and design step.
- the detrending parameters (Fig. 3.4.D): by default, the parameter is set to 'No detrending'. However, we recommend to perform a detrending in the case of time series data such as fMRI (and only in that case). When selecting polynomial, the 'order' parameter will appear, with a default value of 1. Changing this value will increase the order of the polynomial used to fit the data. If 'Discrete Cosine Transform' is selected, the editable parameter corresponds to the cutoff frequency (in seconds) of the high-pass filter. Please note that, when including more than one run ('modality') into a feature set, nothing will prevent the user from using different detrending methods/parameters. We however highly recommend to use a consistent detrending in the same FS.
- the scaling (Fig. 3.4.E): 'no scaling' is the default option. However, when dealing with quantitative modalities such as PET, the user should provide one value per scan, stored in a vector in a .mat file under the variable name 'scaling'.

When working with Graphical User Interfaces (GUIs), some messages might appear in MATLABworkspace. These can display information about the operations currently performed or explain why the toolbox does not do as the user expected (e.g. when a file could not be loaded or if information was input in a wrong format). Therefore we strongly encourage the user to have a look at MATLABprompt when using GUIs.

3.4 Matlab batch interface

The Matlab Batch system allows the input/selection of all parameters and options aforementioned. Just note that the batch is based on the names of the modalities and/or conditions. Therefore, for the batch to work properly, names should be consistent across all steps, starting from data and design to the model specification and running. The hierarchy for the case of a feature set containing one fMRI modality is displayed in (Fig. 3.5).

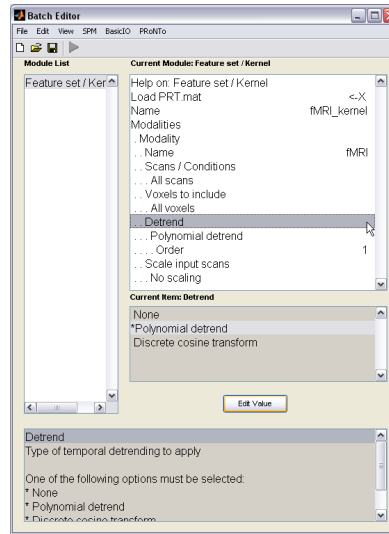


Figure 3.5: Matlabbatch GUI.

Note: Defining all important steps in one batch and running that batch will overwrite the `PRT.mat` previously created and thus delete the links between the `PRT.mat` and the computed kernel(s) and feature set(s). The file arrays would then be recomputed each time the batch is launched. For large datasets, we therefore recommend splitting the batch in two parts: a data and design and prepare feature set part and a second part comprising the model specification, run model and compute weights modules. This would indeed allow changing, e.g. model parameters, without recomputing the feature sets and kernels.

Chapter 4

Model & CV scheme specification

This is where we explain how a model is specified, as well as the cross-validation (CV) scheme necessary to validate the model.

Lots of text to come!

Chapter 5

Mode & feature weights estimation

This is where we explain how we run the model and calculate the weights.
Lots of text to come!

Chapter 6

Results display

This is where we explain how you can display results within the GUI. In particular, what are all the nice results we can show/use to write good papers. Lots of text to come!

Part II

Batching system

Chapter 7

Data & Design

Specify the group(s) of data set.

7.1 Directory

Select a directory where the PRT.mat file containing the specified design and data matrix will be written.

7.2 Groups

Add data and design for one group. Click 'new' or 'repeat' to add another group.

7.2.1 Group

Specify data and design for the group.

Name

Name of the group. Example: 'Controls'.

Select by

Depending on the type of data at hand, you may have many images (scans) per subject, such as a fMRI time series, or you may have many subjects with only one or a small number of images (scans) per subject, such as PET images. If you have many scans per subject select the option 'subjects'. If you have one scan for many subjects select the option 'scans'.

Subjects Add subjects.

Subject Add new modality for this subject.

Modality Add new data modality.

NAME Name of modality. Example: 'BOLD'.

INTERSCAN INTERVAL Specify interscan interval (TR). The units should be seconds.

SCANS Select scans (images) for this modality. They must all have the same image dimensions, orientation, voxel size etc.

DATA & DESIGN Specify data and design.

Load SPM.mat Load design from SPM.mat (if you have previously specified the experimental design with SPM).

Specify design Specify design: scans (data), onsets and durations.

Units for design The onsets of events or blocks can be specified in either scans or seconds.

Conditions Specify conditions. You are allowed to combine both event- and epoch-related responses in the same model and/or regressor. Any number of condition (event or epoch) types can be specified. Epoch and event-related responses are modeled in exactly the same way by specifying their onsets [in terms of onset times] and their durations. Events are specified with a duration of 0. If you enter a single number for the durations it will be assumed that all trials conform to this duration. For factorial designs, one can later associate these experimental conditions with the appropriate levels of experimental factors.

Condition Specify condition: name, onsets and duration.

Name Name of condition.

Onsets Specify a vector of onset times for this condition type.

Durations Specify the event durations. Epoch and event-related responses are modeled in exactly the same way but by specifying their different durations. Events are specified with a duration of 0. If you enter a single number for the durations it will be assumed that all trials conform to this duration. If you have multiple different durations, then the number must match the number of onset times.

Multiple conditions Select the *.mat file containing details of your multiple experimental conditions.

If you have multiple conditions then entering the details a condition at a time is very inefficient. This option can be used to load all the required information in one go. You will first need to create a *.mat file containing the relevant information.

This *.mat file must include the following cell arrays (each 1 x n): names, onsets and durations. eg. names=cell(1,5), onsets=cell(1,5), durations=cell(1,5), then names2='SSent-DSpeak', onsets2=[3 5 19 222], durations2=[0 0 0 0], contain the required details of the second condition. These cell arrays may be made available by your stimulus delivery program, eg. COGENT. The duration vectors can contain a single entry if the durations are identical for all events.

Time and Parametric effects can also be included. For time modulation include a cell array (1 x n) called tmod. It should have a single number in each cell. Unused cells may contain either a 0 or be left empty. The number specifies the order of time modulation from 0 = No Time Modulation to 6 = 6th Order Time Modulation. eg. tmod3 = 1, modulates the 3rd condition by a linear time effect.

For parametric modulation include a structure array, which is up to 1 x n in size, called pmod. n must be less than or equal to the number of cells in the names/onsets/durations cell arrays. The structure array pmod must have the fields: name, param and poly. Each of these fields is in turn a cell array to allow the inclusion of one or more parametric effects per column of the design. The field name must be a cell array containing strings. The field param is a cell array containing a vector of parameters. Remember each parameter must be the same length as its corresponding onsets vector. The field poly is a cell array (for consistency) with each cell containing a single number specifying the order of the polynomial expansion from 1 to 6.

Note that each condition is assigned its corresponding entry in the structure array (condition 1 parametric modulators are in pmod(1), condition 2 parametric modulators are in pmod(2), etc. Within a condition multiple parametric modulators are accessed via each fields cell arrays. So for condition 1, parametric modulator 1 would be defined in pmod(1).name1, pmod(1).param1, and pmod(1).poly1. A second parametric modulator for condition 1 would be defined as pmod(1).name2, pmod(1).param2 and pmod(1).poly2. If there was also a parametric modulator for condition 2, then remember the first modulator for that condition is in cell array 1: pmod(2).name1, pmod(2).param1, and pmod(2).poly1. If some, but not all conditions are parametrically modulated, then the non-modulated indices in the pmod structure can be left blank. For example, if conditions 1 and 3 but not condition 2 are modulated, then specify pmod(1) and pmod(3). Similarly, if conditions 1 and 2 are modulated but there are 3 conditions overall, it is only necessary for pmod to be a 1 x 2 structure array.

EXAMPLE:

Make an empty pmod structure:

```
pmod = struct('name','','param','','poly',);
```

Specify one parametric regressor for the first condition:

```
pmod(1).name1 = 'regressor1';
```

```
pmod(1).param1 = [1 2 4 5 6];
```

```

pmod(1).poly1 = 1;
Specify 2 parametric regressors for the second condition:
pmod(2).name1 = 'regressor2-1';
pmod(2).param1 = [1 3 5 7];
pmod(2).poly1 = 1;
pmod(2).name2 = 'regressor2-2';
pmod(2).param2 = [2 4 6 8 10];
pmod(2).poly2 = 1;

```

The parametric modulator should be mean corrected if appropriate. Unused structure entries should have all fields left empty.

Covariates Select a .mat file containing your covariates (i.e. any other data/information you would like to include in your design). This file should contain a variable 'R' with the covariates.

No design Do not specify design. This option can be used for modalities (e.g. structural scans) that do not have an experimental design.

Scans Select this option if you have many subjects to spatially normalise, but there is one or a small number of scans for each subject.

Modality Specify modality, such as name and data.

Name Name of modality. Example: 'BOLD'.

Files Select scans (images) for this modality. They must all have the same image dimensions, orientation, voxel size etc.

Regression targets (per scans) Enter one regression target per scans. or enter the name of a variable This variable should be a vector [Nscans x 1], where Nsubjs is the number of subjects.

Covariates Select a .mat file containing your covariates (i.e. any other data/information you would like to include in your design). This file should contain a variable 'R' with the covariates.

7.3 Masks

Select first-level (pre-processing) mask for each modality. The name of the modalities should be the same as the ones entered for subjects/scans.

7.3.1 Modality

Specify name of modality and file for each mask.

Name

Name of modality. Example: 'BOLD'.

File

Select one mask for each modality.

7.4 HRF overlap

If using fMRI data please specify the width of the hemodynamic response function (HRF). This will be used to calculate the overlap between events. Leave as 0 for other modalities.

7.5 HRF delay

If using fMRI data please specify the delay of the hemodynamic response function (HRF). This will be used to calculate the overlap between events. Leave as 0 for other modalities.

7.6 Review

Would you like to review the design?.

Chapter 8

Feature set / Kernel

Compute feature set according to the design specified

8.1 Load PRT.mat

Select data/design structure file (PRT.mat).

8.2 Name

Target name for kernel matrix. This should contain only alphanumerical characters or underscores (-).

8.3 Modalities

Add modalities

8.3.1 Modality

Specify modality, such as name and data.

Name

Name of modality. Example: 'BOLD'. Must match design specification

Scans / Conditions

Which task conditions do you want to include in the kernel matrix? Select conditions: select specific conditions from the timeseries. All conditions: include all conditions extracted from the timeseries. All scans: include all scans for each subject. This may be used for modalities with only one scan per subject (e.g. PET), if you want to include all scans from an fMRI timeseries (assumes you have not already detrended the timeseries and extracted task components)

All scans No design specified. This option can be used for modalities (e.g. structural scans) that do not have an experimental design or for an fMRI design where you want to include all scans in the timeseries

All Conditions Include all conditions in this kernel matrix

Voxels to include

Specify which voxels from the current modality you would like to include

All voxels Use all voxels in the design mask for this modality

Specify mask file Select a mask for the selected modality.

Detrend

Type of temporal detrending to apply

None Do not detrend the data

Polynomial detrend Perform a voxel-wise polynomial detrend on the data (1 is linear detrend)

Order Enter the order for polynomial detrend (1 is linear detrend)

Discrete cosine transform Use a discrete cosine basis set to detrend the data.

Cutoff of high-pass filter (second) The default high-pass filter cutoff is 128 seconds (same as SPM)

Scale input scans

Do you want to scale the input scans to have a fixed mean (i.e. grand mean scaling)?

No scaling Do not scale the input scans

Specify from *.mat Specify a mat file containing the scaling parameters for each modality.

Chapter 9

Specify model

Construct model according to design specified

9.1 Load PRT.mat

Select data/design structure file (PRT.mat).

9.2 Model name

Name for model

9.3 Use kernels

Are the data for this model in the form of kernels/basis functions? If 'No' is selected, it is assumed the data are in the form of feature matrices

9.4 Feature sets

Enter the name of a feature set to include in this model. This can be kernel or a feature matrix.

9.5 Model Type

Select which kind of predictive model is to be used.

9.5.1 Classification

Specify classes and machine for classification.

Classes

Specify which elements belong to this class. Click 'new' or 'repeat' to add another class.

Class Specify which groups, modalities, subjects and conditions should be included in this class

Name Name for this class, e.g. 'controls'

Groups Add one group to this class. Click 'new' or 'repeat' to add another group.

Group Specify data and design for the group.

GROUP NAME Name of the group to include. Must exist in PRT.mat

SUBJECTS Subject numbers to be included in this class. Note that individual numbers (e.g. 1), or a range of numbers (e.g. 3:5) can be entered

CONDITIONS / SCANS Which task conditions do you want to include? Select conditions: select specific conditions from the timeseries. All conditions: include all conditions extracted from the timeseries. All scans: include all scans for each subject. This may be used for modalities with only one scan per subject (e.g. PET), if you want to include all scans from an fMRI timeseries (assumes you have not already detrended the timeseries and extracted task components)

Specify Conditions Specify the name of conditions to be included

Condition Specify condition:.

Name Name of condition to include.

All Conditions Include all conditions in this model

All scans No design specified. This option can be used for modalities (e.g. structural scans) that do not have an experimental design or for an fMRI design where you want to include all scans in the timeseries

Machine

Choose a prediction machine for this model

SVM Classification Binary support vector machine.

Arguments Arguments for `prt_machine_svm_bin`. You should use `-t 4` if you selected 'use kernels' option, and `-t 0` otherwise. See libSVM documentation for details.

Gaussian Process Classification Gaussian Process Classification

Arguments Arguments for `prt_machine_gpml`.

Random Forest Random Forest. Breiman, Leo (2001). "Random Forests".

Machine Learning 45:5-32. This is a wrapper around Peter Geurt's implementation in his Regression Tree package.

Ntrees Number of trees in the forest.

Custom machine Choose another prediction machine

Function Choose a function that will perform prediction.

Arguments Arguments for prediction machine.

9.5.2 Regression

Add group data and machine for regression.

Groups

Add one group to this regression model. Click 'new' or 'repeat' to add another group.

Group Specify data and design for the group.

Group name Name of the group to include. Must exist in PRT.mat

Subjects Subject numbers to be included in this class. Note that individual numbers (e.g. 1), or a range of numbers (e.g. 3:5) can be entered

Modality name Name of modality. We only allow one modality for regression model per group at this moment

Example: 'BOLD'. Must match design specification

Machine

Choose a prediction machine for this model

Kernel Ridge Regression Kernel Ridge Regression.

Regularization Regularization for `prr_machine_krr`.

Relevance Vector Regression Relevance Vector Regression. Tipping, Michael E.; Smola, Alex (2001).

"Sparse Bayesian Learning and the Relevance Vector Machine". Journal of Machine Learning Research 1: 211?244.

Custom machine Choose another prediction machine

Function Choose a function that will perform prediction.

Arguments Arguments for prediction machine.

9.6 Cross-validation type

Choose the type of cross-validation to be used

9.6.1 Leave one subject out

Leave a single subject out each cross-validation iteration

9.6.2 Leave one subject per group out

Leave out a single subject from each group at a time. Appropriate for repeated measures or paired samples designs.

9.6.3 Leave one block out

Leave out a single block or event from each subject each iteration. Appropriate for single subject designs.

9.6.4 Leave one run/session out

Leave out a single run (modality) from each subject each iteration. Appropriate for single subject designs with multiple runs/sessions.

9.6.5 Custom

Load a cross-validation matrix. Note that an interface will be provided for this functionality in a later release

9.7 Data operations

Specify operations to apply

9.7.1 Mean centre features

Select an operation to apply.

9.7.2 Other Operations

Include other operations?

No operations

No design specified. This option can be used for modalities (e.g. structural scans) that do not have an experimental design or for an fMRI design where you want to include all scans in the timeseries

Select Operations

Add zero or more operations to be applied to the data before the prediction machine is called. These are executed within the cross-validation loop (i.e. they respect training/test independence) and will be executed in the order specified.

Operation Select an operation to apply.

Chapter 10

Run model

Trains and tests the predictive machine using the cross-validation structure specified by the model.

10.1 Load PRT.mat

Select PRT.mat (file containing data/design structure).

10.2 Model name

Name of a model. Must match your entry in the
'Specify model' batch module.

Part III

Data processing examples

Chapter 11

Data set 1

This is where we explain how how to process data set 1.

Lots of text to come!

Chapter 12

Data set 2

This is where we explain how how to process data set 2.

Lots of text to come!

Part IV

Advanced topics

Chapter 13

PRT structure

This is how the main PRT structure is organised.
PRT

- group
 - gr_name
 - subject
 - subj_name()
 - modality()
 - mod_name
 - TR
 - scans
 - design
 - conds
 - cond_name()
 - onsets()
 - durations()
 - rt_trial()
 - scans()
 - blocks()
 - discardedscans()
 - hrfdiscardedscans()
 - stats
 - overlap
 - goodscans
 - discscans
 - meanovl
 - stdovl
 - mgoodovl
 - sgoodovl
 - goodovl
 - TR
 - unit
 - covar
- masks
 - mod_name

- fname
- fs
 - fs_name
 - k_file
 - id_col_names
 - fas
 - im
 - ifa
 - modality
 - mod_name
 - detrend
 - param_dt
 - mode
 - idfeat_fas
 - normalise
 - type
 - scaling
 - id_mat
- fas
 - mod_name
 - dat
 - detrend
 - param_dt
 - hdr
 - fname
 - dim
 - mat
 - pinfo
 - dt
 - n
 - descrip
 - private
 - idfeat_img
- model
 - model_name()
 - input()
 - use_kernel
 - type
 - machine
 - function
 - args
 - class
 - class_name()
 - group()
 - gr_name

- subj
 - num()
 - modality()
- fs
 - fs_name
- samp_idx
- targets
- targ_allscans
- cv_mat
- operations
- cv_type
- output()
 - fold
 - targets()
 - predictions()
 - stats()
 - con_mat
 - acc
 - c_acc
 - b_acc
 - acc_lb
 - acc_ub
 - func_val()
 - type()
 - alpha()
 - b()
 - totalSV()
- stats
 - con_mat
 - acc
 - c_acc
 - b_acc
 - acc_lb
 - acc_ub

Chapter 14

List of PRoNTo functions

This is the list of PRoNTo functions, including the subdirectories: `machines` and `utils`.

14.1 `prt.m`

Pattern Recognition for Neuroimaging Toolbox, PRoNTo.

This function initializes things for PRoNTo and provides some low level functionalities

14.2 `prt_apply_operation.m`

function to apply a data operation to the training, test and

```
in.train:      training data
in.tr_id:      id matrix for training data
in.use_kernel: are the data in kernelised form
in.tr_targets: training targets (optional field)
in.pred_type:  'classification' or 'regression' (required for tr_targets)
```

A test set may also be specified, which require the following fields:

```
in.test:      test data
in.testcov:   test covariance (only if use_kernel = true)
in.te_targets: test targets
in.te_id:     id matrix for test data
```

`opid` specifies the operation to apply, where:

- 1 = Temporal Compression
- 2 = Sample averaging (average samples for each subject/condition)
- 3 = Mean centre features over subjects
- 4 = Divide data vectors by their norm
- 5 = Perform a GLM (fMRI only)

N.B: - all operations are applied independently to training and test partitions
- see Chu et. al (2011) for mathematical descriptions of operations

1 and 2 and Shawe-Taylor and Cristianini (2004) for a description of operation 3.

References:

Chu, C et al. (2011) Utilizing temporal information in fMRI decoding: classifier using kernel regression methods. *Neuroimage*. 58(2):560-71.
 Shawe-Taylor, J. and Cristianini, N. (2004). *Kernel methods for Pattern analysis*. Cambridge University Press.

14.3 prt_check_design.m

FORMAT [conds] = prt_check_design(cond,tr,units,hrfoverlap)

Check the design and discards scans which are either overlapping between conditions or which do not respect a minimum time interval between conditions (due to the width of the HRF function).

INPUT

- cond : structure containing the names, durations and onsets of the conditions
- tr : interscan interval (TR)
- units : 1 for seconds, 0 for scans
- hrfoverlap : value to correct for BOLD overlap (in seconds)
- hrfdelay : value to correct for BOLD delay (in seconds)

OUTPUT

the same cond structure containing supplementary fields:

- scans : scans retained for further classification
- discardedscans: scans discarded because they overlapped between conditions
- hrfdiscardedscans: scans discarded because they didn't respect the minimum time interval between conditions
- blocks: represents the grouping of the stimuli (for cross-validation)
- stats: struct containing the original time intervals, the time interval with only the 'good' scans, their means and standard deviation

14.4 prt_compute_weights.m

FORMAT prt_compute_weights(PRT,in)

This function calls prt_weights to compute weights

14.5 prt_cv_model.m

Function to run a cross-validation structure on a given model

Inputs:

PRT containing the specified model plus the following arguments:

in.fname: filename for PRT.mat (string)

`in.model_name`: name for this model (string)

Outputs:

Writes the following fields in the PRT data structure:

```
PRT.model(m).output.fold(i).targets:    targets for fold(i)
PRT.model(m).output.fold(i).predictions: predictions for fold(i)
PRT.model(m).output.fold(i).stats:      statistics for fold(i)
PRT.model(m).output.fold(i).custom:     optional fields
```

Notes: - The `PRT.model(m).input` fields are set by `prt_init_model`, not by this function

14.6 prt_cv_opt_param.m

Function to pass optional parameters into the classifier. This is primarily used for complex data prediction methods that need to know something about the experimental design that is normally not accessible to generic prediction functions (e.g. task onsets or TR). Examples of this kind of classifier include multi-class classifier using kernel regression (MCKR) and the machine that implements nested cross-validation.

Inputs:

```
PRT: main data structure
ID:  id matrix for the current cross-validation fold
CV:  cross-validation structure (current fold only)
```

Outputs:

Provides the following fields for use by the classifier

```
param.cv_fold:  the vector from the CV matrix controlling this CV fold
.id_fold:      the id matrix for this fold
.group:        group data for subjects included in this fold (copied
               from PRT)
.onsets:       matrix of event onsets (n_events x n_conditions)
.durations:    matrix of event durations (n_events x n_conditions)
.TR:          TR for this group/subject/modality/event
.unit:         units for this design 0 = scans, 1 = seconds
.ids:         reduced id matrix (group,subject,modality)
```

N.B.: - The `onsets`, `durations`, `TR`, `Unit` and `ids` fields will only be populated for subjects that have a design specified that has the same number of events for all subjects included. This should probably be improved in the future.

14.7 prt_data_conditions.m

PRT_DATA_CONDITIONS M-file for `prt_data_conditions.fig`

PRT_DATA_CONDITIONS, by itself, creates a new PRT_DATA_CONDITIONS or raises the existing singleton*.

`H = PRT_DATA_CONDITIONS` returns the handle to a new `PRT_DATA_CONDITIONS` or the handle to the existing singleton*.

`PRT_DATA_CONDITIONS('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_DATA_CONDITIONS.M` with the given input arguments.

`PRT_DATA_CONDITIONS('Property','Value',...)` creates a new `PRT_DATA_CONDITIONS` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_data_conditions_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_data_conditions_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.8 `prt_data_modality.m`

`PRT_DATA_MODALITY` M-file for `prt_data_modality.fig`

`PRT_DATA_MODALITY`, by itself, creates a new `PRT_DATA_MODALITY` or raises the existing singleton*.

`H = PRT_DATA_MODALITY` returns the handle to a new `PRT_DATA_MODALITY` or the handle to the existing singleton*.

`PRT_DATA_MODALITY('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_DATA_MODALITY.M` with the given input arguments.

`PRT_DATA_MODALITY('Property','Value',...)` creates a new `PRT_DATA_MODALITY` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_data_modality_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_data_modality_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.9 `prt_data_review.m`

`PRT_DATA_REVIEW` M-file for `prt_data_review.fig`

`PRT_DATA_REVIEW`, by itself, creates a new `PRT_DATA_REVIEW` or raises the existing singleton*.

`H = PRT_DATA_REVIEW` returns the handle to a new `PRT_DATA_REVIEW` or the handle to the existing singleton*.

PRT_DATA_REVIEW('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_DATA_REVIEW.M with the given input arguments.

PRT_DATA_REVIEW('Property','Value',...) creates a new PRT_DATA_REVIEW or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_data_review_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_data_review_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.10 prt_defaults.m

Sets the defaults which are used by PRT, Pattern Recognition in Neuroimaging Toolbox

FORMAT prt_defaults

This file can be customised to any the site/person own setup. Individual users can make copies which can be stored on their own matlab path. Make sure your 'prt_defaults' is the first one found in the path. See matlab documentation for details on setting path.

Care must be taken when modifying this file!

The structure and content of this file are largely inspired by SPM:
<http://www.fil.ion.ucl.ac.uk/spm>

14.11 prt_fs.m

Function to build file arrays containing the (linearly detrended) data and compute a linear (dot product) kernel from them

Inputs:

in.fname: filename for the PRT.mat (string)
 in.fs_name: name of fs and relative path filename for the kernel matrix

in.mod(m).mod_name: name of modality to include in this kernel (string)
 in.mod(m).detrend: detrend (scalar: 0 = none, 1 = linear)
 in.mod(m).param_dt: parameters for the kernel detrend (e.g. DCT bases)
 in.mod(m).mode: 'all_cond' or 'all_scans' (string)
 in.mod(m).mask: mask file used to create the kernel
 in.mod(m).normalise: 0 = none, 1 = normalise_kernel, 2 = scale modality
 in.mod(m).matnorm: filename for scaling matrix

Outputs:

Calls prt_init_fs to populate basic fields in PRT.fs(f)...

Writes PRT.mat
 Writes the kernel matrix to the path indicated by in.fs_name

14.12 prt_func2html.m

Script to generate the list of .m functions into html files
 which can be browsed around with your favourite browser.

Note that this script relies on the M2HTML package which is **NOT**
 distributed with PRoNTTo!

For more information, please read the M2HTML tutorial and FAQ at:
 \$<\$<http://www.artefact.tk/software/matlab/m2html/>>\$

14.13 prt_get_defaults.m

Get/set the defaults values associated with an identifier

FORMAT defaults = prt_get_defaults
 Return the global "defaults" variable defined in prt_defaults.m.

FORMAT defval = prt_get_defaults(defstr)
 Return the defaults value associated with identifier "defstr".
 Currently, this is a '.' subscript reference into the global
 "prt_def" variable defined in prt_defaults.m.

FORMAT prt_get_defaults(defstr, defval)
 Sets the defaults value associated with identifier "defstr". The new
 defaults value applies immediately to:
 * new modules in batch jobs
 * modules in batch jobs that have not been saved yet
 This value will not be saved for future sessions of PRoNTTo. To make
 persistent changes, edit prt_defaults.m.

The structure and content of this file are largely inspired by SPM &
 Matlabbatch.

<http://www.fil.ion.ucl.ac.uk/spm>
<http://sourceforge.net/projects/matlabbatch/>

14.14 prt_get_filename.m

out = prt_get_filename(ids)

14.15 prt_init_fs.m

function to initialise the kernel data structure

FORMAT: Two modes are possible:

```

fid = prt_init_fs(PRT, in)
[fid, PRT, tocomp] = prt_init_fs(PRT, in)

```

USAGE 1:

function will return the id of a feature set or an error if it doesn't exist in PRT.mat

Input:

in.fs_name: name for the feature set (string)

Output:

fid : is the identifier for the feature set in PRT.mat

USAGE 2:

function will create the feature set in PRT.mat and overwrite it if it already exists.

Input:

in.fs_name: name for the feature set (string)

in.fname: name of PRT.mat

in.mod(m).mod_name: name of the modality

in.mod(m).detrend: type of detrending

in.mod(m).mode: 'all_scans' or 'all_cond'

in.mod(m).mask: mask used to create the feature set

in.mod(m).param_dt: parameters used for detrending (if any)

in.mod(m).normalise: scale the input scans or not

in.mod(m).matnorm: mat file used to scale the input scans

Output:

fid : is the identifier for the model constructed in PRT.mat

Populates the following fields in PRT.mat (copied from above):

PRT.fs(f).fs_name

PRT.fs(f).fas

PRT.fs(f).k_file

Also computes the following fields:

PRT.fs(f).id_mat: Identifier matrix (useful later)

PRT.fs(f).id_col_names: Columns in the id matrix

Note: this function does not write PRT.mat. That should be done by the calling function

14.16 prt_init_model.m

function to initialise the model data structure

FORMAT: Two modes are possible:

```
mid = prt_init_model(PRT, in)
```

```
[mid, PRT] = prt_init_model(PRT, in)
```

USAGE 1:

 function will return the id of a model or an error if it doesn't
 exist in PRT.mat

Input:

in.model_name: name of the model (string)

Output:

mid : is the identifier for the model in PRT.mat

USAGE 2:

 function will create the model in PRT.mat and overwrite it if it
 already exists.

Input:

in.model_name: name of the model to be created (string)
 in.use_kernel: use kernel or basis functions for this model (boolean)
 in.machine: prediction machine to use for this model (struct)
 in.type: 'classification' or 'regression'

Output:

Populates the following fields in PRT.mat (copied from above):
 PRT.model(m).input.model_name
 PRT.model(m).input.type
 PRT.model(m).input.use_kernel
 PRT.model(m).input.machine

Note: this function does not write PRT.mat. That should be done by the
 calling function

14.17 prt_latex.m

Extract information from the toolbox m-files and output them as usable
 .tex files which can be directly included in the manual.

There are 2 types of m2tex operations:

1. converting the job configuration tree, i.e. *_cfg_* files defining the
 batching interface into a series of .tex files.
 NOTE: Only generate .tex files for each exec_branch of prt_batch.
2. converting the help header of the functions into .tex files.

These files are then included in a manually written prt_manual.tex file,
 which also includes chapter/sections written manually.

File derived from that of the SPM8 distribution.
<http://www.fil.ion.ucl.ac.uk/spm>

14.18 prt_load.m

Function to load the PRT.mat and check its integrity regarding the kernels and feature sets that it is supposed to contain. Updates the set feature name if needed.

input : name of the PRT.mat, path included

output : PRT structure updated

14.19 prt_load_blocks.m

Load one or more blocks of data.

This script is a effectively a wrapper function that for the routines that actually do the work (SPM nifti routines)

The syntax is either:

`img = prt_load_blocks(filenamees, block_size, block_range)` just to specify continuous blocks of data

or

`img = prt_load_blocks(filenamees, voxel_index)` to access non continuous blocks

14.20 prt_model.m

Function to configure and build the PRT.model data structure

Input:

`model.fs(f).fs_name:` feature set(s) this CV approach is defined for
`model.fs(f).fs_features:` feature selection mode ('all' or 'mask')
`model.fs(f).mask_file:` mask for this feature set (fs_features='mask')

`in.fname:` filename for PRT.mat
`in.model_name:` name for this cross-validation structure
`in.type:` 'classification' or 'regression'
`in.use_kernel:` does this model use kernels or features?
`in.operations:` operations to apply before prediction

`in.fs(f).fs_name:` feature set(s) this CV approach is defined for

`in.class(c).class_name`
`in.class(c).group(g).subj(s).num`
`in.class(c).group(g).subj(s).modality(m).mod_name`
 EITHER: `in.class(c).group(g).subj(s).modality(m).conds(c).cond_name`
 OR: `in.class(c).group(g).subj(s).modality(m).all_scans`
 OR: `in.class(c).group(g).subj(s).modality(m).all_cond`

`in.cv.type:` type of cross-validation ('loso', 'logso', 'custom')
`in.cv.mat_file:` file specifying CV matrix (if type='custom');

Output:

This function performs the following functions:

1. populates basic fields in PRT.model(m).input
2. computes PRT.model(m).input.targets based on in.class(c)...
3. computes PRT.model(m).input.samp_idx based on targets
4. computes PRT.model(m).input.cv_mat based on the labels and CV spec

14.21 prt_normalise_kernel.m

FORMAT K_normalised = prt_normalise_kernel(K)

This function normalises the kernel matrix such that each entry is divided by the product of the std deviations, i.e.

$K_{\text{new}}(x,y) = K(x,y) / \sqrt{\text{var}(x)*\text{var}(y)}$

14.22 prt_permutation.m

Function to compute permutation test

Inputs:

PRT: PRT structured including model

n_permu: number of permutations

modelid: model ID

Outputs:

for classification

permutation.c_acc: Permuted accuracy per class

permutation.b_acc: Permuted balanced accuracy

permutation.pvalue.b_acc: p-value for c_acc

permutation.pvalue.c_acc: p-value for b_acc

for regression

permutation.corr: Permuted correlation

permutation.mse: Permuted mean square error

permutation.corr: p-value for corr

permutation.mse: p-value for mse

14.23 prt_preproc.m

Function to preprocess the images, by loading each one of them (or the ones corresponding to the selected scans when a design was specified), applying the masks on them and, if asked, detrend along each voxel along the time series.

INPUT:

fname filename and path to PRT.mat

OUTPUT:

results are saved on disk.

14.24 prt_remove_confound.m

```
[Kr, R] = prt_remove_confound(K,C)
```

Function to remove confounds from kernel.

14.25 prt_stats.m

Function to compute predictions machine performance statistics

Inputs:

model.predictions: predictions derived from the predictive model

model.type: what type of prediction machine (e.g. 'classifier','regression')

t: true targets

flag: 'fold' for statistics in each fold

'model' for statistics in each model

Outputs:

Classification:

stats.con_mat: Confusion matrix (nClasses x nClasses matrix, pred x true)

stats.acc: Accuracy (scalar)

stats.b_acc: Balanced accuracy (nClasses x 1 vector)

stats.c_acc: Accuracy by class (nClasses x 1 vector)

stats.c_pv: Predictive value for each class (nClasses x 1 vector)

Regression:

stats.mse: Mean square error between test and prediction

stats.corr: Correlation between test and prediction

14.26 prt_struct2latex.m

Function that takes in a structure S and writes down the latex code describing the whole structure and substructures recursively.

The routine specifically generates the 'adv_PRTstruct.tex' file that is included, in the prt_manual.

Bits of the code and copied/inspired by spm_latex.m from the SPM8 distribution: <http://www.fil.ion.ucl.ac.uk/spm>

14.27 prt_text_input.m

PRT_TEXT_INPUT M-file for prt_text_input.fig

PRT_TEXT_INPUT, by itself, creates a new PRT_TEXT_INPUT or raises the

existing singleton*.

H = PRT_TEXT_INPUT returns the handle to a new PRT_TEXT_INPUT or the handle to the existing singleton*.

PRT_TEXT_INPUT('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_TEXT_INPUT.M with the given input arguments.

PRT_TEXT_INPUT('Property','Value',...) creates a new PRT_TEXT_INPUT or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_text_input_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_text_input_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.28 prt_ui_compute_weights.m

PRT_UI_COMPUTE_WEIGHTS M-file for prt_ui_compute_weights.fig

PRT_UI_COMPUTE_WEIGHTS, by itself, creates a new PRT_UI_COMPUTE_WEIGHTS or raises the existing singleton*.

H = PRT_UI_COMPUTE_WEIGHTS returns the handle to a new PRT_UI_COMPUTE_WEIGHTS or the handle to the existing singleton*.

PRT_UI_COMPUTE_WEIGHTS('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_COMPUTE_WEIGHTS.M with the given input arguments.

PRT_UI_COMPUTE_WEIGHTS('Property','Value',...) creates a new PRT_UI_COMPUTE_WEIGHTS or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_compute_weights_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_compute_weights_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.29 prt_ui_cv_model.m

PRT_UI_CV_MODEL M-file for prt_ui_cv_model.fig

PRT_UI_CV_MODEL, by itself, creates a new PRT_UI_CV_MODEL or raises the existing singleton*.

H = PRT_UI_CV_MODEL returns the handle to a new PRT_UI_CV_MODEL or the handle to the existing singleton*.

PRT_UI_CV_MODEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_CV_MODEL.M with the given input arguments.

PRT_UI_CV_MODEL('Property','Value',...) creates a new PRT_UI_CV_MODEL or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_cv_model_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_cv_model_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.30 prt_ui_design.m

PRT_UI_DESIGN M-file for prt_ui_design.fig

PRT_UI_DESIGN, by itself, creates a new PRT_UI_DESIGN or raises the existing singleton*.

H = PRT_UI_DESIGN returns the handle to a new PRT_UI_DESIGN or the handle to the existing singleton*.

PRT_UI_DESIGN('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_DESIGN.M with the given input arguments.

PRT_UI_DESIGN('Property','Value',...) creates a new PRT_UI_DESIGN or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_design_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_design_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.31 prt_ui_kernel_construction.m

PRT_UI_KERNEL MATLAB code for prt_ui_kernel.fig

PRT_UI_KERNEL, by itself, creates a new PRT_UI_KERNEL or raises the existing singleton*.

H = PRT_UI_KERNEL returns the handle to a new PRT_UI_KERNEL or the handle to the existing singleton*.

PRT_UI_KERNEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_KERNEL.M with the given input arguments.

`PRT_UI_KERNEL('Property','Value',...)` creates a new `PRT_UI_KERNEL` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_kernel_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.32 `prt_ui_main.m`

`PRT_UI_MAIN` M-file for `prt_ui_main.fig`

`PRT_UI_MAIN`, by itself, creates a new `PRT_UI_MAIN` or raises the existing singleton*.

`H = PRT_UI_MAIN` returns the handle to a new `PRT_UI_MAIN` or the handle to the existing singleton*.

`PRT_UI_MAIN('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_UI_MAIN.M` with the given input arguments.

`PRT_UI_MAIN('Property','Value',...)` creates a new `PRT_UI_MAIN` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_main_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_main_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: `GUIDE`, `GUIDATA`, `GUIHANDLES`

14.33 `prt_ui_model.m`

`PRT_UI_KERNEL_CONSTRUCTION` M-file for `prt_ui_kernel.construction.fig`

`PRT_UI_KERNEL_CONSTRUCTION`, by itself, creates a new `PRT_UI_KERNEL_CONSTRUCTION` or raises the existing singleton*.

`H = PRT_UI_KERNEL_CONSTRUCTION` returns the handle to a new `PRT_UI_KERNEL_CONSTRUCTION` or the handle to the existing singleton*.

`PRT_UI_KERNEL_CONSTRUCTION('CALLBACK',hObject,eventData,handles,...)` calls the local function named `CALLBACK` in `PRT_UI_KERNEL_CONSTRUCTION.M` with the given input arguments.

`PRT_UI_KERNEL_CONSTRUCTION('Property','Value',...)` creates a new `PRT_UI_KERNEL_CONSTRUCTION` or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_construction_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All

inputs are passed to `prt_ui_kernel_construction_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.34 `prt_ui_prepare_data.m`

PRT_UI_KERNEL MATLAB code for `prt_ui_kernel.fig`

PRT_UI_KERNEL, by itself, creates a new PRT_UI_KERNEL or raises the existing singleton*.

H = PRT_UI_KERNEL returns the handle to a new PRT_UI_KERNEL or the handle to the existing singleton*.

PRT_UI_KERNEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_KERNEL.M with the given input arguments.

PRT_UI_KERNEL('Property','Value',...) creates a new PRT_UI_KERNEL or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_kernel_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.35 `prt_ui_prepare_datamod.m`

PRT_UI_KERNEL_MODALITY M-file for `prt_ui_kernel_modality.fig`

PRT_UI_KERNEL_MODALITY, by itself, creates a new PRT_UI_KERNEL_MODALITY or raises the existing singleton*.

H = PRT_UI_KERNEL_MODALITY returns the handle to a new PRT_UI_KERNEL_MODALITY or the handle to the existing singleton*.

PRT_UI_KERNEL_MODALITY('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_KERNEL_MODALITY.M with the given input arguments.

PRT_UI_KERNEL_MODALITY('Property','Value',...) creates a new PRT_UI_KERNEL_MODALITY or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before `prt_ui_kernel_modality_OpeningFcn` gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to `prt_ui_kernel_modality_OpeningFcn` via `varargin`.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.36 prt_ui_results.m

PRT_UI_RESULTS MATLAB code for prt_ui_results.fig

PRT_UI_RESULTS, by itself, creates a new PRT_UI_RESULTS or raises the existing singleton*.

H = PRT_UI_RESULTS returns the handle to a new PRT_UI_RESULTS or the handle to the existing singleton*.

PRT_UI_RESULTS('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_RESULTS.M with the given input arguments.

PRT_UI_RESULTS('Property','Value',...) creates a new PRT_UI_RESULTS or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_results_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_results_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.37 prt_ui_reviewCV.m

PRT_UI_REVIEWCV M-file for prt_ui_reviewCV.fig

PRT_UI_REVIEWCV, by itself, creates a new PRT_UI_REVIEWCV or raises the existing singleton*.

H = PRT_UI_REVIEWCV returns the handle to a new PRT_UI_REVIEWCV or the handle to the existing singleton*.

PRT_UI_REVIEWCV('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_REVIEWCV.M with the given input arguments.

PRT_UI_REVIEWCV('Property','Value',...) creates a new PRT_UI_REVIEWCV or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_reviewCV_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_reviewCV_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.38 prt_ui_reviewmodel.m

PRT_UI_REVIEWMODEL M-file for prt_ui_reviewmodel.fig

PRT_UI_REVIEWMODEL, by itself, creates a new PRT_UI_REVIEWMODEL or raises the existing singleton*.

H = PRT_UI_REVIEWMODEL returns the handle to a new PRT_UI_REVIEWMODEL or the handle to the existing singleton*.

PRT_UI_REVIEWMODEL('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_REVIEWMODEL.M with the given input arguments.

PRT_UI_REVIEWMODEL('Property','Value',...) creates a new PRT_UI_REVIEWMODEL or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_reviewmodel_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_reviewmodel_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.39 prt_ui_select_class.m

PRT_UI_SELECT_CLASS M-file for prt_ui_select_class.fig

PRT_UI_SELECT_CLASS, by itself, creates a new PRT_UI_SELECT_CLASS or raises the existing singleton*.

H = PRT_UI_SELECT_CLASS returns the handle to a new PRT_UI_SELECT_CLASS or the handle to the existing singleton*.

PRT_UI_SELECT_CLASS('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_SELECT_CLASS.M with the given input arguments.

PRT_UI_SELECT_CLASS('Property','Value',...) creates a new PRT_UI_SELECT_CLASS or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_select_class_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_select_class_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.40 prt_ui_select_reg.m

PRT_UI_SELECT_REG M-file for prt_ui_select_reg.fig

PRT_UI_SELECT_REG, by itself, creates a new PRT_UI_SELECT_REG or raises the existing singleton*.

H = PRT_UI_SELECT_REG returns the handle to a new PRT_UI_SELECT_REG or the handle to the existing singleton*.

PRT_UI_SELECT_REG('CALLBACK',hObject,eventData,handles,...) calls the local function named CALLBACK in PRT_UI_SELECT_REG.M with the given input arguments.

PRT_UI_SELECT_REG('Property','Value',...) creates a new PRT_UI_SELECT_REG or raises the existing singleton*. Starting from the left, property value pairs are applied to the GUI before prt_ui_select_reg_OpeningFcn gets called. An unrecognized property name or invalid value makes property application stop. All inputs are passed to prt_ui_select_reg_OpeningFcn via varargin.

*See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one instance to run (singleton)".

See also: GUIDE, GUIDATA, GUIHANDLES

14.41 machines

14.41.1 machines\prt_KRR.m

w = prt_KRR(K,t,reg)

14.41.2 machines\prt_machine.m

Run machine function for classification or regression

FORMAT output = prt_machine(d,m)

Inputs:

- d - structure with information about the data, with fields:
 - Mandatory fields:
 - .train - training data (cell array of matrices of row vectors, each [Ntr x D]). each matrix contains one representation of the data. This is useful for approaches such as multiple kernel learning.
 - .test - testing data (cell array of matrices row vectors, each [Nte x D])
 - .tr_targets - training labels (for classification) or values (for regression) (column vector, [Ntr x 1])
 - .use_kernel - flag, is data in form of kernel matrices (true) or in form of features (false)
 - Optional fields: the machine is responsible for dealing with this optional fields (e.g. d.testcov)
- m - structure with information about the classification or regression machine to use, with fields:
 - .function - function for classification or regression (string)

.args - function arguments (either a string, a matrix, or a struct). This is specific to each machine, e.g. for an L2-norm linear SVM this could be the C parameter

Output:

output - output of machine (struct).

Mandatory fields:

.predictions - predictions of classification or regression
[Nte x D]

Optional fields: the machine is responsible for returning parameters of interest. For exemple for an SVM this could be the number of support vector used in the hyperplane weights computation

14.41.3 machines\prt_machine_RT_bin.m

Run binary Ensemble of Regression Tree - wrapper for Pierre Geurt's RT code

FORMAT output = prt_machine_RT_bin(d,args)

Inputs:

d - structure with data information, with mandatory fields:

- .train - training data (cell array of matrices of row vectors, each [Ntr x D]). each matrix contains one representation of the data. This is useful for approaches such as multiple kernel learning.
- .test - testing data (cell array of matrices row vectors, each [Nte x D])
- .tr_targets - training labels (for classification) or values (for regression) (column vector, [Ntr x 1])
- .use_kernel - flag, is data in form of kernel matrices (true) or in form of features (false)

args - vector of RT arguments

- args(1) - number of trees (default: 501)

Output:

output - output of machine (struct).

* Mandatory fields:

.predictions - predictions of classification or regression [Nte x D]

* Optional fields:

.func_val - value of the decision function

.type - which type of machine this is (here, 'classifier')

14.41.4 machines\prt_machine_gpml.m

Run binary SVM - wrapper for libSVM

FORMAT output = prt_machine_gpml(d,args)

Inputs:

d - structure with data information, with mandatory fields:

- .train - training data (cell array of matrices of row vectors, each [Ntr x D]). each matrix contains one representation of the data. This is useful for approaches such as multiple kernel learning.
- .test - testing data (cell array of matrices row vectors, each [Nte x D])
- .testcov - testing covariance (cell array of matrices row vectors, each [Nte x Nte])
- .tr_targets - training labels (for classification) or values (for regression) (column vector, [Ntr x 1])

```

    .use_kernel - flag, is data in form of kernel matrices (true) of in
                  form of features (false)
args      - argument string, where
    -h      - optimise hyperparameters (otherwise don't)
    -l lik   - likelihood function. Currently only lik = 'erf' is
                  supported
    -c cov    - covariance function:
                  'lin' - simple dot product (no hyperparameters)
                  'linb' - dot product with bias (one hyperparameter)

```

Output:

```

output - output of machine (struct).
* Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
* Optional fields:
    .type        - which type of machine this is (here, 'classifier')
    .p           - predictive probabilities
    .loghyper    - log hyperparameters
    .nlml        - negative log marginal likelihood
    .alpha       - GP weighting coefficients
    .sW          - likelihood matrix (see Rasmussen & Williams, 2006)
    .L           - Cholesky factor

```

14.41.5 machines\prt_machine_krr.m

Kernel ridge regression

FORMAT output = prt_machine_svm_bin(d,args)

Inputs:

```

d      - structure with data information, with mandatory fields:
    .train    - training data (cell array of matrices of row vectors,
                  each [Ntr x D]). each matrix contains one representation
                  of the data. This is useful for approaches such as
                  multiple kernel learning.
    .test     - testing data (cell array of matrices row vectors, each
                  [Nte x D])
    .tr_targets - training labels (for classification) or values (for
                  regression) (column vector, [Ntr x 1])
    .use_kernel - flag, is data in form of kernel matrices (true) of in
                  form of features (false)
args    - libSVM arguments

```

Output:

```

output - output of machine (struct).
* Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
* Optional fields:
    .func_val - value of the decision function
    .type     - which type of machine this is (here, 'classifier')

```

14.41.6 machines\prt_machine_rvr.m

Relevance vector regression (training and testing)

FORMAT output = prt_machine_svm_bin(d,args)

Inputs:

```

d      - structure with data information, with mandatory fields:
    .train    - training data (cell array of matrices of row vectors,

```



```

        each [Ntr x D]). each matrix contains one representation
        of the data. This is useful for approaches such as
        multiple kernel learning.
    .test      - testing data (cell array of matrices row vectors, each
                [Nte x D])
    .tr_targets - training labels (for classification) or values (for
                regression) (column vector, [Ntr x 1])
    .use_kernel - flag, is data in form of kernel matrices (true) or in
                form of features (false)
    args      - libSVM arguments
Output:
    output - output of machine (struct).
    * Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
    * Optional fields:
    .func_val - value of the decision function
    .type      - which type of machine this is (here, 'classifier')

```

14.41.7 machines\prt_machine_svm_bin.m

```

Run binary SVM - wrapper for libSVM
FORMAT output = prt_machine_svm_bin(d,args)
Inputs:
    d      - structure with data information, with mandatory fields:
    .train  - training data (cell array of matrices of row vectors,
                each [Ntr x D]). each matrix contains one representation
                of the data. This is useful for approaches such as
                multiple kernel learning.
    .test   - testing data (cell array of matrices row vectors, each
                [Nte x D])
    .tr_targets - training labels (for classification) or values (for
                regression) (column vector, [Ntr x 1])
    .use_kernel - flag, is data in form of kernel matrices (true) or in
                form of features (false)
    args    - libSVM arguments
Output:
    output - output of machine (struct).
    * Mandatory fields:
    .predictions - predictions of classification or regression [Nte x D]
    * Optional fields:
    .func_val - value of the decision function
    .type      - which type of machine this is (here, 'classifier')

```

14.41.8 machines\prt_rvr.m

Optimisation for Relevance Vector Regression

```

[w,alpha,beta,ll]=spm_rvr(Phi,t)
Phi  - MxM matrix derived from kernel function of vector pairs
t    - the values to be matched
w    - weights
alpha - 1/variance for the prior part of the model
beta  - 1/variance for the likelihood part of the model
ll    - the negative log-likelihood.

```

```
[w,alpha,beta,nu,ll]=spm_rvr(K,t,opt)
K      - a cell-array of MxM dot-product matrices.
t      - the values to be matched
opt    - either 'Linear' or 'Gaussian RBF'
          'Linear'      is for linear regression models, where
                        the optimal kernel is generated by
                        [nu(1)*K1 + nu(1)*K2... ones(size(K1,1),1)]
          'Gaussian RBF' is for regression using Gaussian radial basis
                        functions. The kernel is generated from
                        P1 = nu(1)*K1 + nu(1)*K2 ... ;
                        P2 = repmat(diag(P1) ,1,size(P1,2)) +...
                        repmat(diag(P1)',size(P1,1),1) - 2*P1;
                        Phi = exp([-0.5*P2 ones(size(P1,1),1)]);
w      - weights
alpha  - 1/variance for the prior part of the model
beta   - 1/variance for the likelihood part of the model
nu     - parameters that convert the dot-product matrices into
        a kernel matrix (Phi).
ll     - the negative log-likelihood.
```

The first way of calling the routine simply optimises the weights. This involves estimating a restricted maximum likelihood (REML) solution, which maximises $P(\alpha, \beta | t, \Phi)$. Note that REML is also known as Type II Maximum Likelihood (ML-II). The ML-II solution tends towards infinite weights for some the regularisation terms (i.e. $1/\alpha(i)$ approaches 0). The appropriate columns are removed from the model when this happens.

The second way of calling the routine also estimates additional input scale parameters as described in Appendix C of Tipping (2001). This method is much slower, as a full optimisation for the scale parameters is done after each update of the alphas and beta.

see: <http://research.microsoft.com/mlp/RVM/relevance.htm>

Refs:

The Relevance Vector Machine.

In S. A. Solla, T. K. Leen, and K.-R. Mller (Eds.),
Advances in Neural Information Processing Systems 12,
pp. 652-658. Cambridge, Mass: MIT Press.

Michael E. Tipping

Sparse Bayesian Learning and the Relevance Vector Machine
Journal of Machine Learning Research 1 (2001) 211-244

14.41.9 machines\prt_weights.m

Run function to compute weights

FORMAT weights = prt_weights(d,m)

Inputs:

```
d      - data structure
m      - machine structure
        .function - function to compute weights (string)
        .args     - function arguments
```

Output:

weights - weights vector [Nfeatures x 1]

14.41.10 machines\prt_weights_bin_linkkernel.m

Run function to compute weights for linear kernel binary classifiers

FORMAT weights = prt_weights_bin_linkkernel (d,args)

Inputs:

d - data structure
 .datamat - data matrix [Nfeatures x Nexamples]
 .coeffs - coefficients vector [Nexamples x 1]
 args - function arguments (can be empty)

Output:

weights - vector with weights [Nfeatures x 1]

14.41.11 machines\prt_weights_svm_bin.m

Run function to compute weights for binary SVM

FORMAT weights = prt_weights_svm_bin (d,args)

Inputs:

d - data structure
 .datamat - data matrix [Nfeatures x Nexamples]
 .coeffs - coefficients vector [Nexamples x 1]
 args - function arguments (can be empty)

Output:

weights - vector with weights [Nfeatures x 1]

14.42 utils

14.42.1 utils\prt_centre_kernel.m

This function centres the kernel matrix, respecting the independence of training and test partitions. See Shawe-Taylor and Cristianini for background on this approach.

Shawe-Taylor, J. and Cristianini, N. (2004). Kernel methods for Pattern analysis. Cambridge University Press.

14.42.2 utils\prt_checkAlphaNumUnder.m

check whether a given string is alphanumerical or underscore

FORMAT out = prt_checkAlphaNumUnder(s)

Inputs:

s - a string of arbitrary length to check

Output:

out - logical 1 if the all chars in the string are alphanumerical
 logical 0 otherwise

Based on isalpha_num in the identification toolbox

14.42.3 utils\prt_normalise_kernel.m

This function normalises the kernel matrix such that each entry is

divided by the product of the std deviations, i.e.
 $K_{\text{new}}(x,y) = K(x,y) / \sqrt{\text{var}(x)\text{var}(y)}$

Part V

Bibliography

Bibliography

- [1] Christopher M. Bishop. *Pattern Recognition and Machine learning*. Springer, 2006.
- [2] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, April 2006.
- [3] Janaina Mourao-Miranda, Karl J Friston, and Michael Brammer. Dynamic discrimination analysis: a spatial-temporal svm. *Neuroimage*, 36(1):88–99, May 2007.
- [4] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. Adaptive Computation and Machine Learning. the MIT Press,, 2006.